

FRC Scouting App — Project Handoff Summary (v7)

A complete, self-contained record of the offline FRC scouting desktop application build: goals, source material, architecture, what has been implemented and verified in every stage through Stage 13, and exactly what a future session needs to know to continue with no access to any original chat history.

This document supersedes the earlier v6 handoff PDF — it contains everything that PDF did, brought up to date, plus one new stage delivered in a follow-up session: **Stage 13**, a startup-performance optimization (no GUI or calculation-output changes) that replaced the single `scipy.stats.norm.pdf()` call in the curve-fit with an equivalent closed-form function using the standard-library math module, removed two pip dependencies (numpy, pandas) that were never actually imported anywhere in the codebase, and made matplotlib's import lazy in both chart-drawing tabs so an empty/fresh install doesn't pay its import cost at startup. See Section 18 for the full write-up. If this document and the code ever disagree, treat the code as ground truth and update this document.

1. Project Goal

Ian is building a fully offline Python desktop application that replicates the logic of a complex FRC (FIRST Robotics Competition) scouting Google Spreadsheet as a standalone tool. Requirements:

- 100% offline at runtime — no network calls, no external API, no cloud database.
- Runs on both Windows and Mac using only standard, freely-licensed components.
- Local storage only (SQLite file on disk).
- Four required UI tabs, in this order: Match Lookup, Team Lookup, Pick List, Rankings — matching the first four tabs of the source Google Sheet, whose content must appear on the matching tabs.
- The remaining ~28 sheets in the source spreadsheet contain the backend calculation logic and must be replicated exactly, not approximated casually — the user has explicitly and repeatedly stressed correctness over speed (“make no mistakes”, “it is imperative this code is also correctly implemented”).
- The scoring/forecasting logic implements a custom, non-Monte-Carlo skewed-distribution statistical method for forecasting robot performance and predicting match outcomes, documented in a PDF titled “Scouting Algorithm Overview” (a.k.a. `Scouting_Algorithm_Overview_4.pdf`).

2. Source Material

2.1 The Google Sheet

Original file: “Copy of Worlds Scouting Sheet” (Google Sheets ID `1RcHI_oUH3XHg3i4FAZgDR5eB0Wuq5HtgWGzZIrreO7c`). It contains 32 tabs and roughly 215,687 individual formula/value cells. The first four tabs share the exact names of the four required UI tabs; the other 28 tabs are backend calculation sheets.

2.2 How the formulas were extracted

Google Sheets formulas do not survive a plain .xlsx export reliably (Google-only functions like QUERY, ARRAYFORMULA, IMPORTRANGE, FILTER, XLOOKUP, VSTACK can be flattened or dropped). To get every formula as durable plain text, an Apps Script was written and run directly inside the spreadsheet (Extensions → Apps Script) that:

- Iterates every sheet and every used cell, recording Sheet name, Cell (A1 notation), Value (display value), and Formula (raw formula text, or blank for non-formula cells) into a new tab called `FORMULA_EXPORT`.
- Is resumable — because Apps Script has a ~6-minute execution limit, and this workbook has 215k+ cells, the script checkpoints its progress (current sheet index, row index, output row) in Script Properties every 500 rows and on a 5m20s time budget, so re-running the same function (`exportAllFormulas`) continues exactly where it left off. Helper functions `resetExport()` and `checkExportStatus()` were also provided.

The `FORMULA_EXPORT` tab was then downloaded as CSV:

`Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv` (215,687 data rows, columns: Sheet, Cell, Value, Formula). This CSV is the ground-truth source for every formula translated into Python in this project, and the ground-truth source used to find and verify every bug described in Sections 9, 10, 11, and 14 below. Keep this CSV — it

is required to verify or extend any calculation.

2.3 The algorithm PDF

“Scouting Algorithm Overview” (Scouting_Algorithm_Overview_4.pdf, 17 pages) describes the underlying statistical method in prose/derivation form: fitting a skew-normal-like distribution to each team's historical scores WITHOUT Monte Carlo simulation, by (1) picking a transform ($\sqrt{x}$ / $\log_{10} x$ / reciprocal, or their power-inverses for left-skewed data) that best symmetrizes a team's score history, (2) fitting a normal PDF over the transformed axis using the transformed data's own mean/stddev, (3) scaling the resulting curve so its peak is 1, with a hard cutoff around the 90th–98th percentile and a broadening adjustment for weak teams, and (4) combining three teammates' curves and comparing two alliances' combined curves to estimate win probability. Equation OCR in a couple of spots (e.g. the σ_{adj} approximation) was ambiguous (missing radicals/exponents); ground truth was resolved by cross-checking against the exported formulas (2.2) instead of trusting the OCR text alone.

2.4 A note on a missing sheet

Several formulas in the Team Lookup sheet (team name, weak-to-defense notes, climb-confidence flags) reference a sheet called “Super Scouting” that does not exist anywhere in the 32-sheet export. The user confirmed this sheet is not needed and instructed it be ignored — those specific Team Lookup fields are intentionally left unimplemented. Unchanged since the original build, and unchanged through Stage 13.

3. Delivery Approach: Staged, Not One-Shot

Given the size (32 sheets, 215k+ cells) and the presence of genuinely novel statistical logic (not boilerplate spreadsheet math), the build was explicitly staged rather than attempted in one pass, to avoid silently getting the core algorithm wrong:

- **Stage 1** — the raw data pipeline (10 sheets). Complete and delivered.
- **Stage 2** — the skew-normal curve fitting + defense/compatibility stats (4 more sheets). Complete and delivered.
- **Stage 3** — power rankings, alliance combination, match odds, and the real Pick List / Rankings logic (remaining sheets). Complete and delivered, with one explicitly flagged area needing the user's validation (see §6.5).
- **Stage 4** — Data Import Redesign: replaced the original three-CSV granular input model with a streamlined two-CSV model built around the user's real data format, fixed a zero-collapse bug in the Adjusted Score calculation, and validated the full pipeline end-to-end against the user's actual competition data for the first time. Full detail in Section 19.
- **Stage 5** — Adjusted Score Variance Bug Investigation: three real bugs found and fixed in the import/calculation chain between Score Variance and Adjusted Score Variance (Match Schedule authority during import, an occurrence-indexed lookup quirk in the source sheet's own formulas, and a narrow evidence-based typo-correction feature). Verified against the source spreadsheet's own cached formula values. Full detail in Section 9.
- **Stage 6** — Experiment: at the user's explicit request, Adjusted Score Variance was removed from the calculation pipeline entirely and replaced everywhere with plain Score Variance. The user reviewed the results and decided to keep this change — it is current, active, production behavior. Full detail in Section 10.
- **Stage 7** — Expected Score Bug Investigation: the user reported Match Lookup's expected score coming out massively below real match scores, as if the alliance combination were only picking up one of the three teammates' scores. Root-caused and fixed two distinct, real bugs in calculations/alliance_generator.py — a branch-ordering bug that silently zeroed a teammate's contribution right next to its own peak, and an alliance-level axis that was truncated to the same 1–500 range used for a single robot's score even though a 3-robot combined score routinely exceeds that. Both were confirmed as real by cross-checking FORMULA_EXPORT, and Bug 2 was confirmed to exist in the original Google Sheet itself, not just this port. Validated against all 110 real matches with a recorded score. Full detail in Section 11.
- **Stage 8** — UI Quality-of-Life Improvements: at the user's request, Match Lookup and Team Lookup's read-only dropdown selectors were replaced with type-in number fields (digits only, validated as you type, submitted on Enter/focus-out/a “Go” button), and the Rankings tab gained a “Sort by” dropdown (RPP / ROP / RDP / Distance) so the same table can be re-ordered without hiding any columns. No calculation logic was touched. This stage also recorded the results of the user's first manual validation pass of Match Odds against the live Google Sheet. Full detail in Section 12; validation detail in Section 13.

- **Stage 9** — Bug Fix: New-Competition Import Was Mixing With Old Competition Data. The user tested a different competition's worth of data and found that importing a new competition's Match Schedule/Raw Data CSVs did not replace the previously-loaded competition's data — it merged with it, corrupting the roster and every downstream calculation. Root-caused to the import UI never using the `replace_existing` option both importer functions already supported internally. Fixed with an explicit per-import “Update current competition” vs. “New competition” choice, the latter wiping both input tables first. Full detail in Section 14.
- **Stage 10** — Team Lookup: Score Trend Chart. At the user's request, a line chart was added to the top of the Team Lookup tab, plotting the selected team's chronological Score Variance history. No calculation code changed — a pure visualization of data the pipeline already produced. Full detail in Section 15.
- **Stage 11** — Team Lookup: Panel Cleanup. The user reported the Team Lookup tab no longer fit on screen after Stage 10's chart was added, with no way to scroll. Before adding a scroll feature, three now-redundant/unused stat panels (Score Variance, Skew Adjusted Spread, Defense) were removed at the user's request; the user then confirmed on their own machine that this alone fixed the fit problem, so a scroll feature was never built. Full detail in Section 16.
- **Stage 12** — Match Lookup: Score Distribution Chart. At the user's request, a chart was added below the existing alliance panels on the Match Lookup tab, overlaying red and blue alliance's combined-score curves on one plot so their overlap is visually obvious. No calculation code changed. Full detail in Section 17.
- **Stage 13** — Startup Performance Optimization. At the user's request, profiled and fixed the app's slow initial startup: removed a single expensive `scipy.stats.norm.pdf()` call site from the curve-fit (replaced with an equivalent closed-form function using the standard-library `math` module), removed two pip dependencies (`numpy`, `pandas`) that were never actually imported anywhere in the codebase, and made `matplotlib`'s import lazy in both chart-drawing tabs. No calculation output changed (verified bit-for-bit / floating-point-noise identical) and no GUI behavior changed. Full detail in Section 18.

4. Current File Structure

Delivered as a project folder `frc_scouting_app/` (most recently shared zipped as `frc_scouting_app_15.zip` — the zip shared at the end of the Stage 13 session is the current, authoritative version; if a newer-numbered zip has been shared since, that one is authoritative instead). `frc_scouting_app_11.zip` was the Stage 9 checkpoint (the version described by the v5 PDF); `_12` added Stage 10's chart, `_13` added Stage 11's panel cleanup, `_14` added Stage 12's chart, and `_15` added Stage 13's startup-performance changes (`scipy/numpy/pandas` removal in `calculations/distributions.py` and `requirements.txt`, lazy `matplotlib` imports in `match_lookup_tab.py` and `team_lookup_tab.py`) — each is superseded by the next, with only the one stage's changes as its diff from the previous zip. This is the full, current layout:

- **main.py** — entry point, run this
- **database.py** — SQLite schema (local file, no server). Unaffected by Stages 10–13. Changed in Stage 9: added `clear_active_competition_data()`, which deletes every row from both `match_schedule` and `manual_raw_data` so a new competition's CSVs can be imported without mixing with whatever was loaded before. See Section 14.
- **data_import.py** — CSV importers for the 2 active input tables (`manual_raw_data`, `match_schedule`). Unaffected by Stages 10–13 — see Section 19 for the base format spec, Section 9 for the Stage 5 import-time fixes (Match Schedule authority, evidence-based typo correction), and Section 14 for how Stage 9 wires their existing `replace_existing` parameter up to the UI.
- **requirements.txt** — `matplotlib` only, imported lazily by the two chart tabs (see Section 5). `numpy`, `scipy`, and `pandas` were listed here from the original build through Stage 12, but were removed in Stage 13: `grep` confirmed `numpy` and `pandas` were never actually imported anywhere in the codebase, and `scipy`'s one call site (`scipy.stats.norm.pdf()` in `calculations/distributions.py`) was replaced with an equivalent standard-library implementation. See Section 18.
- **README.md** — setup + full data-import format reference + the Stage 5 bug writeups + the Stage 6 experiment writeup + the Stage 7 expected-score bug writeup + the Stage 8 UI-changes writeup + the Stage 9 import-mixing bug writeup + the Stage 10 Team Lookup chart writeup + the Stage 11 Team Lookup panel-cleanup writeup + the Stage 12 Match Lookup chart writeup + the Stage 13 startup-performance writeup (kept in sync with this PDF's Sections 9, 10, 11, 12, 14, 15, 16, 17, and 18).

- **calculations/** — unchanged by any UI stage (8, 9, 10, 11, or 12) in every file, and unaffected in every file except one by Stage 13. `distributions.py` had its implementation changed in Stage 13 (its single `scipy.stats.norm.pdf()` call replaced with an equivalent standard-library-math function) but its inputs and outputs are unchanged — verified bit-for-bit / floating-point-noise identical against the pre-Stage-13 version (Section 18.5). Unaffected in every file, through every stage including 13: `team_roster.py`, `raw_processing.py`, `matrix_stats.py`, `balls_scored.py`, `score_variance.py`, `skew_methods.py`, `transforms.py`, `defense.py`, `power_rankings.py`, `team_ability.py`, `pick_list.py`, `alliance_generator.py`, `match_odds.py` — see Section 6 for what each recreates. Stages 10 and 12 added charts that read these modules' existing outputs (`score_variance` and `alliance_generator`'s `empirical_curve`, respectively) but call no new functions in `calculations/` and modify none of them.

- **ui/app.py** — main window, menu, notebook of 4 tabs. Unaffected by Stages 10, 11, 12, or 13. Changed in Stage 9: added a new `ImportModeDialog` class (a small modal `Toplevel` shown before every CSV import, asking “Update current competition” vs. “New competition”); `_import()` now calls `database.clear_active_competition_data()` and passes `replace_existing=True` into both importer functions when “New competition” is chosen, and passes `replace_existing=False` (the original merge/upsert behavior) when “Update current competition” is chosen; the import-complete message box now says “Cleared previous competition's data.” at the top whenever replace mode ran.

- **match_lookup_tab.py** — tab 1: Match Lookup (live win odds). Changed in Stage 8: the “Match #:” selector is a `tk.Entry` (digits-only key validation) instead of a read-only `tk.Combobox`, with a “Go” button and `<Return>/<FocusOut>` bindings that both call `_render()`. Changed in Stage 12: a new “Score Distribution” chart was added below the red/blue alliance panels, overlaying both alliances' combined-score curves (`AllianceDistribution.empirical_curve`) via a new `_draw_alliance_chart()` method; `_render()` was restructured so `red_dist/blue_dist` are initialized to `None` before the odds-calculation try block (previously local to it) so the chart can reuse them. Changed in Stage 13: the module-level `matplotlib` imports (`Figure`, `FigureCanvasTkAgg`) were moved from the top of the file into `_draw_alliance_chart()` itself — a code-motion only, no change to what the chart draws or how it looks. The win-odds/expected-score calculation itself is untouched — it still calls `build_alliance_distribution()` + `compute_match_odds()` exactly as in Stage 7, and the chart is a display-only addition on top of the same numbers. See Sections 17 and 18.

- **team_lookup_tab.py** — tab 2: Team Lookup. Changed in Stage 8: the same dropdown-to-text-entry change as Match Lookup. Changed in Stage 10: a new “Score Trend” chart was added at the top of the tab, below the team header and above the stat panels, plotting `self.data.score_variance[team].values` via a new `_draw_score_chart()` method. Changed in Stage 11: the separate “Score Variance”, “Skew Adjusted Spread”, and “Defense” stat panels were removed (redundant with the new chart, or unused/redundant with other panels — see Section 16), and the remaining panel grid was reflowed from 5 rows/7 panels down to 3 rows/4 panels: Auto Balls Scored + Teleop Balls Scored, then Fitted Score Distribution + Average Compatability, then Overall Power (spanning both columns). Changed in Stage 13: same code-motion as `match_lookup_tab.py` — the module-level `matplotlib` imports were moved into `_draw_score_chart()`. See Sections 15, 16, and 18.

- **pick_list_tab.py** — tab 3: Pick List (real ranking logic). Unaffected by Stage 7 through Stage 13.

- **rankings_tab.py** — tab 4: Rankings. Unaffected by Stages 9–13. Changed in Stage 8: a new “Sort by” read-only `tk.Combobox` (values: RPP, ROP, RDP, Distance; default RPP) sits above the table.

- **data/scouting.db** — created automatically on first run

All 32 source sheets have a corresponding Python implementation. Every `.py` file's docstring documents which sheet(s) it recreates and the exact formula it is based on — treat those docstrings as the authoritative, line-by-line spec.

`calculations/alliance_generator.py`'s docstring remains the single best place to start if you need to touch alliance-combination logic again. Stages 10, 11, and 12 did not add or change any docstrings in `calculations/` — only in `ui/team_lookup_tab.py` and `ui/match_lookup_tab.py`, both of which had their module docstrings updated to describe the new charts and, for `team_lookup_tab.py`, the Stage 11 panel removal. Stage 13 added one new function-level docstring (`calculations/distributions.py::_normal_pdf()`, explaining why and how it replaces `scipy.stats.norm.pdf()`) and two short inline comments (one at each new lazy-import site, in `match_lookup_tab.py` and `team_lookup_tab.py`) — it did not change any module-level docstring in `ui/` or `calculations/`.

5. Runtime / Environment

- Language: Python 3.11+ (standard installer from python.org on both platforms).

- GUI: tkinter (ships with the standard Python installer — on Mac, must be the official python.org installer, not a Tk-less Homebrew build).
- Local data store: SQLite via the built-in sqlite3 module — a single file at data/scouting.db.
- Math/visualization dependencies: matplotlib only (added in Stage 10), imported lazily as of Stage 13 (see below). numpy, scipy, and pandas were listed in requirements.txt from the original build through Stage 12, but were removed in Stage 13: neither numpy nor pandas was ever imported anywhere in the codebase, and scipy's sole use — `scipy.stats.norm.pdf()`, called once per team per x-value in the curve-fit — was replaced with an equivalent ~10-line closed-form function using the standard-library math module, verified bit-for-bit / floating-point-noise identical (Section 18.5) and roughly two orders of magnitude faster per call. matplotlib is used only for its TkAgg backend (via `matplotlib.backends.backend_tkagg.FigureCanvasTkAgg`), which draws directly into the already-open tkinter window — it ships as part of the matplotlib wheel and relies on the same tkinter already required for the GUI, so it introduced no new system-level dependency and no network calls. As of Stage 13, matplotlib's imports live inside the two chart-drawing methods that need it (`ui/match_lookup_tab.py::_draw_alliance_chart()`, `ui/team_lookup_tab.py::_draw_score_chart()`) rather than at module level, so a fresh install with no CSV imported yet (nothing to chart) doesn't pay matplotlib's import cost at startup; once a competition's data is loaded, matplotlib is still imported at startup exactly as before, since both tabs draw a chart immediately on the first refresh.
- Zero network calls anywhere in the codebase — confirmed by design, no HTTP/socket libraries are imported. Still true as of Stage 13.
- Setup: `pip install -r requirements.txt` then `python main.py`.

Stage 9 added no new dependencies — the `ImportModeDialog` is built entirely from `tk.Toplevel` and `ttk.Button/ttk.Label`, both already part of the standard tkinter/ttk module. Stage 10 added matplotlib as the project's first new dependency since the original build (for the Team Lookup chart); Stages 11 and 12 added no further dependencies — Stage 12's Match Lookup chart reuses the exact same matplotlib/TkAgg embedding pattern Stage 10 already established. Stage 13 removed three dependencies (numpy, scipy, pandas) that had been present since the original build — see Section 18. matplotlib is now the project's only pip dependency.

6. What Has Been Implemented (Stage-by-Stage Detail)

6.1 Local data model

There are two actively used input tables, `manual_raw_data` and `match_schedule`, populated by two CSV imports (see Section 19 for the full format spec, Section 9 for the Stage 5 import-time fixes, and Section 14 for the Stage 9 fix to how a “new competition” clears old data before import). The original `raw_scouting` and `bps_samples` tables and their import paths still exist in the codebase but are unused legacy code — nothing writes to them anymore, and Stage 9's `clear_active_competition_data()` deliberately does not touch them (see Section 14.3) since they hold no competition-specific data currently in use. Every other one of the 32 sheets is a pure formula sheet with no independent input, so it is recreated as a pure Python calculation each time the pipeline runs — there is no need to store intermediate results in the database.

6.2 Stage 1 — Raw Data Pipeline (10 sheets)

- Team Numbers → `calculations/team_roster.py` — team roster + per-team match counts.
- BPS → `calculations/raw_processing.py::team_bps_averages()` — average of each team's manual samples. [LEGACY, unused since Stage 4]
- Unfiltered Raw Data derived columns (S–AC) → `calculations/raw_processing.py::_augment_unfiltered_rows()`. [LEGACY, unused since Stage 4]
- Raw Data → `calculations/raw_processing.py::build_raw_data()`. [LEGACY, unused since Stage 4 — the ACTIVE equivalent is `build_raw_data_from_manual()`]
- Auto Balls Scored / Teleop Balls Scored → `calculations/balls_scored.py` — per-team chronological pivot (up to 15 matches) with `Min(2nd-smallest)/Max(2nd-largest)/STDEV/AVG/RRP`, using `AVG/(STDEV+1)` (note the +1, unique to these two sheets). Still active, unchanged.
- Score Variance → `calculations/score_variance.py::build_score_variance()` — same per-team chronological pivot pattern over Raw Data's Score column, plus Skew, explicitly capped to each team's first 15 chronological matches (a

Stage 5 fix). Independently verified against the source spreadsheet's cached values at 624/625 (99.8%) exact match. This is the value used everywhere downstream as of Stage 6. Its per-team chronological value list (TeamMatrixRow.values) is also the exact data Stage 10's Team Lookup "Score Trend" chart plots directly (Section 15), and the exact data a separate "Score Variance" text panel used to show before Stage 11 removed it as redundant with that chart (Section 16). The calculation itself is unaffected by either stage, or by Stage 13.

- Raw Data Processing → calculations/score_variance.py::build_raw_data_processing() — per match, per alliance slot: computes each alliance's "performance ratio" (sum of 3 teammates' raw scores / sum of their individual averages), and records BOTH that alliance's own ratio AND the OPPOSING alliance's ratio for that same match (the opposing ratio becomes a defense proxy used later). As of Stage 5, this replicates the source sheet's own occurrence-count-based COUNTIF/XLOOKUP indirection instead of a direct (team, match) lookup, including its quirks when a team has a scheduled match with no scouted row. Still needed as of Stage 13, since its own_alliance_ratio/opponent_alliance_ratio outputs feed Defense Adjustment, Average Compatability, and RDP — unaffected by Stage 11's removal of the Defense text panel, since RDP (used elsewhere, e.g. the Overall Power panel) is computed from this data regardless of whether a dedicated Defense panel displays it.
- Match Schedule "Adjusted" columns (N–S) → also in score_variance.py — apportions each match's actual recorded alliance score to the 3 individual robots, proportional to their (occurrence-indexed) raw scouted score share. Unchanged since the Stage 5 occurrence-indexing fix.
- Adjusted Score Variance → build_adjusted_score_variance() — same pivot pattern over the Adjusted values, plus the 4-way skew-method picker. Still fully implemented and callable, and still verified to be a faithful port of the sheet's own formulas (94.4% exact match once known, non-code data issues are excluded). As of Stage 6, it is simply not called by pipeline.py anymore.
- SQRT Method / Log10 Method / 1/X Method → calculations/skew_methods.py — apply one of 4 transforms to reduce skew. As of Stage 6, takes a score_variance parameter (was adjusted_score_variance).
- Skew Adjusted Spreads → skew_methods.py::build_skew_adjusted_spreads() — for each team, picks whichever of the 4 candidate skew values is closest to zero; that method's transformed data becomes the team's final skew-normalized spread — the direct input to Stage 2's curve fit (Stage 2's curve fit itself had its scipy-based implementation replaced with an equivalent standard-library-math one in Stage 13, with no change to any of these numbers — see Section 18). Stage 11 removed its dedicated Team Lookup display panel (Section 16), since the user said this wasn't something they referenced — the calculation itself is untouched and still runs every pipeline build, feeding Stage 2's curve fit exactly as before.

6.2a Skew-method picker

(used repeatedly throughout the project, unchanged): for a team's data, compute the moment-based skew of the raw values; if it's negative (left-skewed) the four candidate transforms are {raw, x^2 , x^3 , x^4 }; if non-negative, {raw, $\sqrt{x}$, $\log_{10}(x)$, $1/x$ }. Whichever candidate's resulting skew has the smallest absolute value is selected (method 1=raw, 2= $\sqrt{x}$, 3= $\log_{10}(x)$, 4= $1/x$). This exact rule reappears at the alliance level in Stage 3 (with weighted/frequency-based stats instead of raw-array stats).

6.3 Stage 2 — Curve Fitting + Defense/Compatibility (4 sheets)

- Skew Distributions → calculations/distributions.py::build_skew_distributions() — for each team and each whole-number score x from 1 to X_AXIS_MAX (500), evaluates a normal PDF at transform(x) using that team's Skew Adjusted Spreads mean/stdev, with $y=0$ above the team's 95th-percentile raw score (hard cutoff). As of Stage 13, that PDF evaluation is calculations/distributions.py::_normal_pdf() — a private, ~10-line closed-form function using the standard-library math module. It replaced scipy.stats.norm.pdf() for startup-performance reasons (a slow import plus ~125x slower per-call overhead — see Section 18) and was verified to produce bit-for-bit / floating-point-noise-identical output to the function it replaced (Section 18.5). As of Stage 6, this function's first parameter is score_variance (was adjusted_score_variance).
- Normalized Distributions → build_normalized_distributions() — scales each team's raw curve so its peak is 1; teams whose Score Variance mean is at or below the league-wide 27th percentile get their curve's y -values replaced with $\sqrt{y}$ (a broadening adjustment for weak/uncertain teams). Produces each team's mode (x of peak y) as a byproduct. This mode value is what Stage 7's branch-splitting logic in alliance_generator.py keys off of, and this per-team curve is what feeds both Team Lookup's existing "Fitted Score Distribution" panel (unaffected by Stage 11) and, indirectly via

build_alliance_distribution(), Stage 12's Match Lookup chart. This function itself was not touched by Stage 13 — only build_skew_distributions()'s single norm.pdf() call site changed implementation, not any output.

- Defense Calculations → calculations/defense.py::build_defense_calculations() — fits a linear trend (SLOPE) through each team's first 12 chronological raw scores, computes residuals from that trend, and derives a trimmed-mean “defensive variance” (dropping the single worst residual), then min-max-normalizes it across all teams into an RRP. Stage 11 removed the dedicated “Defense” Team Lookup panel that displayed this RRP alongside Defense Adjustment's RRP (Section 16); this calculation is otherwise untouched and still feeds RDP (Section 6.4), which the Overall Power panel still shows.
- Defense Adjustment → defense.py::build_defense_adjustment() — reuses Raw Data Processing's per-match “opponent alliance ratio” list per team with its own Min/Max/skew stats. Same Stage 11 panel-removal note as Defense Calculations above — the calculation is unaffected.
- Average Compatability → defense.py::build_average_compatability() — reuses the “own alliance ratio” list per team, but the sheet intentionally excludes the very first chronological match from all of its stats — replicated exactly. Its Team Lookup panel was kept (not removed) in Stage 11.

6.4 Stage 3 — Power Rankings, Pick List, Alliance Odds

- ROP → calculations/power_rankings.py::build_rop() — a weighted blend of Score Variance RRP and Average Compatability RRP (weights 2:0.5); Auto/Teleop Balls Scored RRP feed only the STDEV denominator, not the weighted average itself.
- RDP → build_rdp() — combines (a) how close a team's average opponent-suppression ratio is to the league-best team's, weight 2, with (b) that team's Defense Calculations RRP, weight 1.
- RPP → build_rpp() — weighted blend of ROP (2) and RDP (1), normalized to the league max; also computes a boolean “defense susceptible” flag and a Euclidean “distance from the ideal (1,1) point” in ROP-RDP space. This is the exact field the Rankings “Distance” sort option (Stage 8) sorts on, ascending, and the “Weak to Defense?” flag Team Lookup's Overall Power panel has always shown — the panel Stage 11 pointed to instead of a separate Defense panel.
- Team Ability Distribution → calculations/team_ability.py — league-wide RPP benchmarks: overall average, top-4 average, top-8 average, an “elims window” average, and “Goal Ally RPP” = average of the top-8 and elims-window averages. This becomes the quality bar for the real Pick List.
- Alliance Builder Filtering → calculations/pick_list.py::build_pick_list() — the real Pick List logic. Teams at/above the Goal Ally RPP bar are ranked by Score Variance AVG/STDEV (a consistency ratio); teams below the bar form a fallback list ranked by ROP. Final list = top tier then fallback tier.
- Alliance Base Generator + Blue/Red Alliance Generator → calculations/alliance_generator.py::build_alliance_distribution() — combines three teammates' Normalized Distributions curves into one alliance-level combined score distribution WITHOUT full convolution, by splitting each teammate's curve into an ascending and descending branch (at its mode) and combining branches rank-by-rank across teammates. Two real bugs in this step were found and fixed in Stage 7 — see Section 11. Its output, AllianceDistribution.empirical_curve, is exactly what Stage 12's Match Lookup chart plots (with display-only smoothing — see Section 17); build_alliance_distribution() itself was not modified in Stage 12 or Stage 13.
- Match Odds → calculations/match_odds.py::compute_match_odds() — for each integer score x , sums $P(\text{alliance A} \geq x) \times P(\text{alliance B} \leq x)$ across all x (and the mirror image for B), then normalizes the two totals against each other into a final win probability; also reports each alliance's expected score (weighted mean of its empirical curve). Unchanged in code since Stage 7, including through Stage 13 — Stage 12's chart visualizes the same empirical_curve inputs this function reads, but does not call or modify compute_match_odds() differently, and every win-probability/expected-score number in the Match Lookup panels still comes from this unmodified function.

6.5 Known Uncertainty — the Blue/Red cross-labeling question — PARTIALLY ADDRESSED (see §13)

The Alliance Generator → Match Odds chain was the single most intricate part of the entire 32-sheet workbook. Internally, the SOURCE SHEET cross-labels its outputs: the tab literally named “Blue Alliance Generator” is actually built from the RED alliance's three combined curves (framed as “the score Blue needs to clear to win”), and vice versa for “Red Alliance

Generator” / Blue’s curves. This cross-labeling could NOT be confirmed with full certainty from static formula analysis alone.

Decision made (unchanged since the original build, through Stage 13): this Python port builds each alliance’s combined distribution directly from its OWN three teammates (self-referential, unambiguous) rather than the sheet’s cross-referenced labeling. This is mathematically equivalent for producing a sound win probability and is far easier to read/maintain, but it is NOT a byte-for-byte guarantee of matching the sheet’s literal implementation if that cross-labeling turns out to matter.

Update from Stage 8 (unaffected by Stages 9–13): the user performed the first real-world comparison of this app’s Match Odds output against the live Google Sheet’s own Match Odds for the same match. The two did not produce identical win percentages. The user’s assessment, made directly and not walked back, was that this is expected given it is “impossible to have a perfect port” of a 215,000-cell workbook, that the app is “able to predict matches well and in line with the level of accuracy expected,” and that the current state is passable. The user explicitly chose to bookmark further investigation rather than dig into the discrepancy now — see Section 13 for the full record of this decision and what should happen next.

6.6 Other Explicit Assumptions / Configurable Constants

- `X_AXIS_MAX = 500` (calculations/distributions.py) — the SINGLE-ROBOT score curve is built over whole numbers 1–500, matching the source sheet exactly. Your original competition’s real season data shows individual-robot 95th-percentile cutoffs as high as ~589, so if Team Lookup curves ever look clipped for your highest-scoring teams, raise this constant. As of Stage 7, this no longer affects the alliance-level combined curve — raising it only affects single-robot curves. Unchanged through Stage 13 (Stage 13 changed this file’s PDF-evaluation implementation, not this constant or any output value it governs).
- `ALLIANCE_X_AXIS_MAX = 3 × X_AXIS_MAX = 1500` (calculations/alliance_generator.py, added in Stage 7) — the 3-ROBOT COMBINED score curve used by Match Lookup (and, as of Stage 12, charted there) is built over whole numbers 1–1500. If `X_AXIS_MAX` is ever raised, this constant will automatically scale with it. Unchanged through Stage 13.
- `ELIM_ALLIANCE_TEAMS = 24` (calculations/team_ability.py) — the source sheet switches its “elims window” between 24 and 32 teams based on a live per-event cell (“Pick List!B2”) whose value wasn’t available during this build. 24 (= 8 alliances × 3, the standard FRC playoff bracket) is the default. Adjust per-event if using a non-standard bracket size. Unchanged.
- Exact-match score lookup in `alliance_generator.py`’s empirical-curve step mirrors the sheet’s VLOOKUP exact-match behavior — integer scores with no exact combined sample land on density 0 rather than being smoothed/interpolated between neighbors. Unchanged by any stage through 13. This is precisely why Stage 12’s Match Lookup chart applies smoothing for DISPLAY PURPOSES ONLY before plotting this curve — see Section 17.3 — without changing the underlying exact-match data itself.

7. Current UI State (all 4 tabs are fully wired, no placeholders)

- **Match Lookup** — type a match number into the “Match #.” field (digits only) and press Enter, click “Go”, or click away from the field. Shows both alliances’ three teams with Score Variance avg/RRP, the actual recorded score, and (built live, not cached) expected score + win probability from calculations/alliance_generator.py + match_odds.py. A blank field prompts “Type a match number above and press Enter.”; an unknown match number shows “Match {n} not found.” As of Stage 12, a “Score Distribution” chart also sits below these panels, overlaying red and blue alliance’s combined-score curves on one plot so their overlap is visually obvious (or, for a match with a missing team’s fitted distribution, a “Score distribution unavailable for this match.” placeholder) — see Section 17. Stage 13 changed only when this chart’s matplotlib import runs, not what it draws — see Section 18.
- **Team Lookup** — same type-in-field pattern as Match Lookup. An unrecognized team number shows “Team {n} not found in the imported roster.” As of Stage 10, a “Score Trend” line chart sits at the top of the tab (below the team header, above the stat panels), plotting the selected team’s chronological Score Variance history with a dashed average line, or a “No scouted score history for this team.” placeholder if there is none. As of Stage 11, the separate “Score Variance”, “Skew Adjusted Spread”, and “Defense” stat panels below the chart were removed as redundant/unused, leaving Auto Balls Scored, Teleop Balls Scored, Fitted Score Distribution, Average Compatability,

and Overall Power — see Sections 15 and 16. As with Match Lookup, Stage 13 changed only this chart's import timing, not its appearance.

- **Pick List** — real Alliance Builder Filtering output: Top Picks (above the Goal Ally RPP bar, sorted by quality ratio) then Other Picks (below the bar, sorted by ROP). Unaffected by any stage from 7 through 13.
- **Rankings** — a “Sort by” dropdown (RPP / ROP / RDP / Distance, default RPP) sits above the table. Changing it instantly re-sorts and re-numbers the same table. Unaffected by Stages 9 through 13.
- **Data menu** — “Import Match Schedule CSV...”, “Import Raw Data CSV (Team + Score Breakdown)...”, a separator, “Set My Team Number...”, “Recalculate Now.” Unaffected by Stages 10–13. As of Stage 9, choosing either import command first opens an ImportModeDialog asking “Update current competition” or “New competition” (or Cancel) before the file is processed — see Section 14 for full detail.

8. Testing Performed So Far

- After every stage, a small hand-built synthetic SQLite dataset was inserted directly via `sqlite3`, and `calculations/pipeline.py::build_all()` was run end-to-end in a headless Python process to confirm it executes without exceptions and produces sane-looking numbers.
- Stage 3 was checked for: RPP/ROP/RDP all landing in a sensible [0,1]-ish range; the Pick List correctly splitting into tiers; Match Odds probabilities for a sample match summing to 1.0 with plausible expected scores.
- Stage 4 validated against the user's REAL competition data for the first time: Match Schedule CSV (110 matches, all imported) and Raw Data CSV (654 data rows, 625 successfully imported, 29 correctly auto-skipped blank/placeholder rows, 0 unresolved-alliance rows at the time), full pipeline run cleanly against it (75 teams, 74-entry Pick List), and a directional Match Odds sanity check for one real match against its actual recorded score.
- Stage 5 root-caused the discrepancy the user reported between the app's Adjusted Score Variance and the live Google Sheet's own calculation by re-importing the user's real CSVs through the production import functions and cross-checking every intermediate value cell-by-cell against `FORMULA_EXPORT`'s own cached values. Final measured results: Score Variance 624/625 (99.8%) exact match; Adjusted Score Variance 561/594 (94.4%) exact match once two confirmed non-code-bug causes are excluded.
- Stage 6 re-ran the full pipeline end-to-end against the same real data after removing Adjusted Score Variance from the calculation chain: 75 teams, 74-entry Pick List (unchanged), Goal Ally RPP recomputed to ≈ 0.780 (was ≈ 0.773 in the original Stage 4 baseline), and a live Match Odds computation for match 1 completed with no errors.
- Stage 7 re-ran the full pipeline end-to-end against the same real data again (no changes to team count, Pick List size, or any upstream calculation): still 75 teams, 74-entry Pick List, confirming no regression. Then, for all 110 real matches with a recorded official score, computed each alliance's expected score BEFORE and AFTER the Stage 7 fixes: mean absolute error between expected and actual alliance score dropped from ≈ 175 points to ≈ 100 points, and win-direction accuracy rose from 67% (74/110) to 69% (76/110).
- Stage 8 syntax-checked every `.py` file (`ast.parse`) after the UI edits, including all three changed `ui/` files — all parsed cleanly. The Tkinter GUI itself was still not visually rendered/smoke-tested in the build environment used for that session (no display available there).
- Stage 9 reproduced the reported bug headlessly before fixing it (`manual_raw_data` going from 625 to 1250 rows on a repeat merge-mode import), then confirmed the fix left it at exactly 625 rows with a matching 75-team/625-row pipeline result, and confirmed the legitimate same-competition merge/upsert use case (`match_schedule` staying at 110 rows on a repeat merge-mode import) was not broken. Every `.py` file re-syntax-checked cleanly. As with Stage 8, the GUI had still not been visually smoke-tested on a real machine as of the end of that session.
- Stage 10 re-ran the full real-data pipeline import (625 imported, 29 skipped, 34 schedule conflicts, 1 typo correction, 75-team roster — matching the established baseline exactly, confirming no regression), then pulled a real team's (1796) Score Variance history (8 matches, values 50.5–305.0, average ≈ 237.2) and rendered the new chart logic headlessly via `matplotlib`'s non-interactive `Agg` backend to a PNG with no errors; the empty-history case (a team with no `score_variance` row) was also confirmed to render its placeholder text instead of raising an exception. Every `.py` file re-syntax-checked cleanly. Unlike Stages 8 and 9, Stage 10's (and Stage 11's) UI changes WERE subsequently confirmed by the user on their own real machine — the user reported “the graph looks good” and, after Stage 11, that “the team lookup tab looks great now” and no scrolling was needed. See Section 15.3 and Section 16.4.

- Stage 11 was a pure removal of three Tkinter widgets and a grid reflow; verified with `ast.parse` across the project and a visual review of the resulting 3-row/4-panel layout. The user's own real-machine test (see above) is this stage's primary validation — they confirmed the decluttered tab fit on screen without needing a scroll feature, which was the explicit goal. See Section 16.4.
- Stage 12 extracted the exact smoothing/trimming logic used by the new `_draw_alliance_chart()` method and ran it against all 110 real matches in the user's Match Schedule — all 110 produced a valid 1500-point smoothed curve for both alliances with zero errors. Three real matches spanning the win-probability range were then rendered headlessly (Agg backend) to PNG for visual inspection: an 18.9%/81.1% match showed the two curves clearly separated with modest overlap, a near-coinflip 50.9%/49.1% match showed them almost fully overlapping, and a 75.7%/24.3% match also rendered without error — exactly the visual behavior the chart is meant to show. Every .py file re-syntax-checked cleanly. UNLIKE Stages 10 and 11, Stage 12's Match Lookup chart has NOT yet been confirmed by the user on their own real machine as of this writing.
- Stage 13 profiled the app's import chain with Python's `-X importtime` flag and diffed `calculations.pipeline.build_all()`'s full output against the same real Match Schedule/Raw Data CSVs, both before and after removing `scipy` from the curve-fit: `skew_distribution_curves` matched to floating-point noise (max absolute difference `1.1368683772161603e-13` across every team/x-value pair), `normalized_distributions'` `curves/mode_x/upper_cutoff` matched similarly (max absolute difference `4.440892098500626e-16`, with `mode_x` and `upper_cutoff` bit-for-bit identical), and `pick_list / rpp` — the most downstream outputs any of this feeds — were bit-for-bit identical before vs. after. Timing: `build_all()` alone dropped from 0.63s to 0.02s; the combined cost of importing `calculations.pipeline`, initializing the database, and running `build_all()` once (everything Stage 13 could affect, excluding `tkinter/matplotlib`) dropped from ≈ 1.30 s to ≈ 0.04 s. Every .py file was re-compiled with `py_compile` and all compiled cleanly. Like Stages 8 and 9 (and like Stage 12, still open), no visual/GUI smoke test was possible in the build environment used for this session — it has no display and does not have `tkinter` installed at all. See Section 18.6.

9. Stage 5 — Adjusted Score Variance Bug Investigation & Fixes

Symptom reported by the user: Adjusted Score Variance, as calculated by the app, differed from what the live Google Sheet calculated for the same data. Score Variance (the step immediately before it) was confirmed correct by the user's own testing. This meant the bug had to be somewhere in the chain between Score Variance and Adjusted Score Variance: Raw Data Processing, the Match Schedule "Adjusted" columns, or Adjusted Score Variance's own pivot.

Method: the user's real CSVs (`Untitled_spreadsheet_-_Raw_Data.csv`, `Untitled_spreadsheet_-_Match_Schedule.csv`) were imported through the actual production import functions into a scratch SQLite database, and every intermediate value the pipeline produced was compared cell-by-cell against

`Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv`'s own cached Value column for the matching Sheet/Cell.

9.1 Bug 1 — Match Schedule not treated as authoritative during Raw Data import

Root cause: `import_raw_data_csv()` used to trust each row's own Alliance/Position cell before checking the already-imported `match_schedule` table, and would unconditionally upsert that (sometimes unreliable) value into `match_schedule`. In the user's real export, the Position column does not reliably reflect the true official Red1-3/Blue1-3 slot. A concrete example: match 87's true roster (Red: 10935, 88, 3044 / Blue: 1540, 2783, 6436, per the authoritative Match Schedule CSV) was scrambled into (Red: 88, 6436, 2783 / Blue: 3354, 10935, 2783) purely because several Raw Data rows' own Position cells disagreed with the schedule.

Fix: `import_raw_data_csv()` now always tries a `match_schedule` lookup for a row's team+match first, and only falls back to that row's own Alliance/Position cell if the lookup fails. The `match_schedule` upsert is now only performed in that fallback case.

9.2 Bug 2 — Score Variance not capped to 15 matches

`build_score_variance()` did not cap each team's chronological score history to 15 matches, while the source sheet's Score Variance tab only has 15 match columns (B:P). This matters because Raw Data Processing's occurrence-indexed lookup (Bug 3 below) depends on this cap matching exactly. Fix: `build_score_variance()` now truncates each team's value list to its first 15 chronological matches.

9.3 Bug 3 — the occurrence-indexed lookup (a real quirk of the source sheet, now replicated)

The sheet's Raw Data Processing columns do NOT look up “this team's raw score for this match” by a direct (team, match) key. The actual formula is: `COUNTIF({$A$3:$C<row>,$K$3:$M<row>}, <team>) → XLOOKUP(that count, Score Variance's header row 1–15, that team's Score Variance row)`. In words: “how many times has this team appeared anywhere in Match Schedule's 6 team columns, from the top of the schedule through this row?” gives an occurrence index K, and the sheet then takes the K-th value from that team's Score Variance history (chronological, capped at 15).

Confirmed as a real, reproducible behavior of the source sheet itself by tracing a specific example: team 687 is scheduled for 9 matches but has only 8 Raw Data rows (missing match 51); at match 99 (its 8th scheduled appearance), the sheet's own cached Raw Data Processing value for team 687 (70.19661809) turned out to exactly equal team 687's real score from match 108 — its 8th actually scouted match. Fix: `build_raw_data_processing()` was rewritten to replicate this occurrence-indexed lookup exactly.

9.4 A fourth issue found via user feedback — evidence-based typo correction

After Bug 1's fix shipped, the user reported that Team Lookup now showed a “phantom” team, 325, that wasn't a real team. Investigation traced this to match 47: the Match Schedule CSV listed team 325 as Blue 3, while the Raw Data CSV had a row for team 3256 at that same match/slot. Team 325 appeared exactly once with zero corroborating raw data anywhere; team 3256 appeared in 8 other matches with consistent, real data. The user independently confirmed this was a typo (“325” being “3256” with the trailing digit dropped).

Fix implemented (and confirmed correct): `import_raw_data_csv()` now checks, for every conflict it detects, whether the team currently on record is a total “orphan” (appears exactly once, zero corroborating raw-data rows anywhere) while the conflicting row's team is well-established (at least 3 other rows for it elsewhere in the same CSV). Only in that narrow, lopsided case is the schedule slot corrected automatically; every more ambiguous conflict (34 found in the user's real data) is left unresolved-but-reported via `schedule_conflicts` / `schedule_conflict_examples`. Corrections are always reported via `likely_typo_corrections` / `likely_typo_correction_examples`, never silent.

An important caution surfaced during this work: matching a cached ground-truth snapshot and being factually correct are not always the same thing when the snapshot itself was generated from imperfect source data. Always check the most authoritative source available (here, the user directly) before concluding an automated “fix” is wrong just because it diverges from a cache.

9.5 Validation results

With all of the above fixes in place, importing the user's real CSVs produces: 625 Raw Data rows imported, 29 correctly skipped as blank/placeholder rows, 0 unresolved-alliance rows, 34 `schedule_conflicts` (correctly left unresolved and reported), and 1 `likely_typo_corrections` entry (match 47: 325 → 3256, confirmed correct by the user). The full pipeline then produces exactly 75 teams and a 74-entry Pick List. Score Variance 624/625 (99.8%) exact match; Adjusted Score Variance 564/622 (90.7%) exact match overall, rising to 561/594 (94.4%) once two confirmed non-code-bug causes are excluded.

9.6 Code changes summary (Stage 5)

`data_import.py` — `import_raw_data_csv()`: Match Schedule lookup now tried before the row's own Alliance/Position cell; `match_schedule` upsert only performed on fallback; added the evidence-based typo-correction check; return dict extended with `schedule_conflicts`, `schedule_conflict_examples`, `likely_typo_corrections`, `likely_typo_correction_examples`. `calculations/score_variance.py` — `build_score_variance()` truncates to 15 matches; `build_raw_data_processing()` rewritten for the occurrence-indexed lookup. `ui/app.py` — post-import message box extended. `README.md` — new sections documenting all three fixes.

10. Stage 6 — Experiment: Adjusted Score Variance Removed

10.1 Rationale

At the user's explicit request. Real scouting data is human-entered and will always have holes in it — exactly the conditions Adjusted Score Variance's occurrence-indexed lookup (Section 9.3) is most sensitive to, since a single missing scouted match for a team silently shifts every later lookup for that team. Score Variance, by contrast, is simpler and was independently verified at 99.8% exact match. The user's own testing on the live Google Sheet found Adjusted Score Variance was only ever a minor adjustment on top of Score Variance.

Current status: the user has reviewed the results of this experiment and decided to keep the change. This is current, active, production behavior — not a proposal. Unaffected by any stage from 7 through 13.

10.2 What changed

In `calculations/pipeline.py`: `build_all()` no longer calls `score_variance.build_adjusted_score_variance()` at all. Every downstream call that used to receive the Adjusted Score Variance dict now receives the plain Score Variance dict (sv) instead. The `ComputedData` dataclass no longer has an `adjusted_score_variance` field at all (fails loudly rather than silently, by design). In the UI: `ui/match_lookup_tab.py`'s per-team panel reads `self.data.score_variance`; its “Adj. Avg Score” column header was renamed to “Avg Score”. `ui/team_lookup_tab.py`'s separate “Adjusted Score Variance” panel was removed entirely. All of this remains true after the Stage 8 through Stage 13 UI/backend edits — none of them touched what these panels display, and Stage 11's later removal of the plain “Score Variance” panel (Section 16) is a separate, distinct change from this Stage 6 removal of the “Adjusted” one.

10.3 What did NOT change

`calculations/score_variance.py::build_raw_data_processing()` still runs, unchanged — its `own_alliance_ratio` and `opponent_alliance_ratio` outputs are what Defense Adjustment, Average Compatability, and RDP are built from, and none of that is Adjusted Score Variance. `build_adjusted_score_variance()` itself was not deleted — it's simply unused by `pipeline.py` now, and remains fully callable/testable in isolation.

10.4 How to revert

A small, localized change: in `calculations/pipeline.py`, add back `asv = score_variance.build_adjusted_score_variance(rdp_rows_raw)` and pass `asv` (instead of `sv`) into the five functions listed in 10.2; add `adjusted_score_variance` back to `ComputedData`; in `ui/team_lookup_tab.py` and `ui/match_lookup_tab.py`, point the removed/renamed panels back at `self.data.adjusted_score_variance`. This path is also documented in `README.md` and `pipeline.py`'s own module docstring. It is unaffected by the Stage 8 selector changes, the Stage 9 import-mode changes, the Stage 10–12 chart/panel changes, or Stage 13's startup-performance changes — note that Team Lookup's plain “Score Variance” panel this section originally referred to no longer exists as of Stage 11 (see Section 16), so reverting this experiment today would mean re-adding an “Adjusted Score Variance” panel where the (now-removed) “Score Variance” panel used to be, rather than restoring a panel that currently exists.

10.5 A concrete before/after data point

Running the full pipeline against the user's real, Stage-5-corrected data (75 teams, `my_team_num=1796`): the Team Ability Distribution's Goal Ally RPP bar computed to approximately 0.836 immediately after the Stage 5 fixes (Adjusted Score Variance still active downstream), and approximately 0.780 after the Stage 6 change — compared to approximately 0.773 in the original Stage 4 baseline run. Team count (75) and Pick List size (74) were identical across all runs, and remain identical through Stage 13 as well (none of the later stages touches Team Ability Distribution / Pick List).

11. Stage 7 — Expected Score Bug Investigation & Fixes

11.1 Symptom reported

The user reported that Match Lookup's expected score was coming out massively below real match scores, and specifically noted it looked “as if the alliance combination were only picking up one of the three teammates' scores.”

11.2 Investigation method

The investigation focused on `calculations/alliance_generator.py::build_alliance_distribution()`, the function that combines three teammates' Normalized Distributions curves into one alliance-level combined-score distribution. Every step was cross-checked directly against `FORMULA_EXPORT`'s “Alliance Base Generator”, “Blue Alliance Generator”, and “Match Odds” sheets — reading the actual formula text cell-by-cell.

11.3 Bug 1 — the ascending branch was combined in the wrong point order

Each teammate's normalized curve is unimodal: it rises to a peak of 1 at its mode, then falls. The sheet splits each teammate's curve at its own mode into two monotonic branches, then combines all three teammates' branches rank-by-rank. An earlier version of this port built the ascending branch in plain left-to-right (x-ascending) order. Tracing the sheet's own formulas revealed this is backwards: the sheet collects the ascending-branch values in natural x order, then

passes them through `SORT(values, SEQUENCE(n), FALSE())` — sorting by an increasing 1..n key in descending order, which simply REVERSES the list. With the branch order backwards, a teammate whose ascending branch was shorter than its teammates' got zero-padded starting right next to its OWN mode — its most heavily-weighted, most-likely -score region. Fixed by reversing the ascending branch's point order in `_split_branches()` to match the sheet.

11.4 Bug 2 — the alliance-level combined curve was capped at the same 500-point axis as a single robot

Both this port and the ORIGINAL Google Sheet build the combined-alliance empirical curve over the exact same 1–500 axis used for a single robot's score. But a 3-robot combined score can run up to 3x a single robot's max, and in this season's real data it very often does. Confirmed two independent ways from `FORMULA_EXPORT`: (1) "Alliance Base Generator"'s own cached combined-sample "Point Value" columns run as high as ~1079 with a median around ~670; (2) the user's actual recorded Match Schedule scores (110 matches, 220 alliance-scores) range from 129 to 819 with a median of 439, and 78 of 220 (35%) exceed 500 outright. Fixed by giving the alliance-level curve its own, separate, wider axis: `ALLIANCE_X_AXIS_MAX = 3 × X_AXIS_MAX` (currently 1500). Single-robot curves (feeding Team Lookup) are completely unaffected.

11.5 Validation results

- Mean absolute error between expected and actual alliance score: ≈175 points before → ≈100 points after.
- Win-direction accuracy: 67% (74/110) before → 69% (76/110) after.
- Team roster (75 teams) and Pick List size (74 entries) were identical before and after.
- Worked example, match 1 (red: 4253, 4099, 6436 vs. blue: 1796, 5549, 604; actual recorded scores red 263 / blue 642): expected scores went from R287/B335 to R412/B545 (correctly and much more strongly favoring blue, in line with the lopsided actual result).

11.6 Code changes summary (Stage 7)

`calculations/alliance_generator.py` — `_split_branches()` now reverses the ascending branch's point order after collecting it. A new module-level constant, `ALLIANCE_X_AXIS_MAX = 3 * X_AXIS_MAX`, was added and `empirical_curve` construction inside `build_alliance_distribution()` now ranges over `X_AXIS_MIN..ALLIANCE_X_AXIS_MAX` instead of `X_AXIS_MIN..X_AXIS_MAX`. No other files needed changes.

11.7 An important caveat for anyone comparing against the live Google Sheet

Bug 2 (the axis truncation) is present in the original Google Sheet itself, confirmed directly from its own cached `FORMULA_EXPORT` values — not just in this port. This means the fix applied here is a deliberate, intentional divergence from literal sheet-quirk fidelity, made because the un-fixed behavior produced results that were clearly less accurate relative to real match outcomes. If you ever compare this app's Match Lookup numbers directly against the live Google Sheet for the same match, expect the live sheet's own expected-score-style figures to still show the same underestimation this app used to show before Stage 7 — that is the sheet behaving as designed, not a sign this app is wrong. This is directly relevant to Section 13's Stage 8 validation note — some amount of percentage disagreement between this app and the live sheet is expected and, for Bug 2 specifically, this app's number should be treated as the more accurate one. It is also directly relevant to Stage 12's chart (Section 17): the `empirical_curve` this app plots already reflects the wider, more accurate `ALLIANCE_X_AXIS_MAX=1500` axis, so the chart will show probability mass out past 500 that the live sheet's own internal curve would not.

12. Stage 8 — UI Quality-of-Life Improvements

At the user's request, three small, purely-presentational changes were made in this stage. No calculation code was touched anywhere in `calculations/`, `data_import.py`, `database.py`, or `pipeline.py` — every change in this stage lived entirely inside `ui/match_lookup_tab.py`, `ui/team_lookup_tab.py`, and `ui/rankings_tab.py`.

12.1 Match Lookup and Team Lookup: dropdown → type-in field

The `ttk.Combobox(state="readonly")` was replaced with a `ttk.Entry` bound to the same `StringVar`, with digits-only input validation via Tk's `validatecommand` mechanism (`self.register(self._validate_digits)` registers a Python callback as a Tcl command, wired up with `validate="key"` so it runs on every keystroke). Both `<Return>` and `<FocusOut>` are bound to call the tab's existing `_render()` method. A "Go" button was added next to each field. `refresh()` no longer populates a

combobox values list — it seeds the field with the first known number only if the field is currently empty. `_render()` in both tabs now handles three states explicitly: an empty field, a value not found in the data, and the normal found case.

12.2 Rankings: added a Sort-by dropdown

A new read-only `ttk.Combobox` labeled “Sort by:” was added above the existing `Treeview`, with values RPP / ROP / RDP / Distance and a default of RPP. A class-level `SORT_OPTIONS` dict maps each label to a (attribute_name, descending) tuple — RPP/ROP/RDP sort descending (bigger is better), Distance sorts ascending (smaller — closer to the ideal (1,1) point — is better). Selecting a new option fires `<<ComboboxSelected>>`, which calls `_render()`, which re-sorts `self.data.rpp.values()` and re-numbers Rank 1..N accordingly. All seven columns are still populated for every row regardless of sort choice.

12.3 Testing performed for Stage 8

Every `.py` file in the project was re-syntax-checked with `ast.parse()` after these edits and all parsed cleanly. The logic was traced by hand against tkinter's documented `validatecommand/%P` substitution behavior and `ttk.Treeview`'s insert/delete semantics. The GUI had still never been visually smoke-tested in the build environment used at the time (no display available there) as of the end of Stage 8 — this remained a standing caveat going into Stage 9 as well; see Section 19.

13. Google Sheet Validation — First Pass Complete, Bookmarked for Follow-Up

This is the long-standing open item from Section 6.5: comparing this app's Match Odds output against the live Google Sheet's Match Odds for the same real match. Unaffected by Stage 9 through Stage 13.

13.1 What was done

During the Stage 8 session, the user independently compared this app's Match Lookup prediction output against the live Google Sheet's own Match Odds calculation for at least one real match. This was done by the user directly (outside of any tool available in the build environment used at the time), so no specific match number, the exact percentages compared, or a sample size is captured in this handoff.

13.2 What was found

The percentages produced by this app and by the live Google Sheet were different, not identical. The user's conclusion was that this is most likely attributable to the inherent, expected differences between a from-scratch Python port and a 215,000-cell spreadsheet — in the user's own words, it is “impossible to have a perfect port” of a workbook this large and intricate. The user judged the app to be “able to predict matches well and in line with the level of accuracy expected,” and called the current state “passable for now.”

13.3 Decision made

The user explicitly chose not to dig further into the discrepancy right then, and instead asked to bookmark it: as more competitions' data becomes available and gets tested through the app, any systematic pattern in how far off the predictions run should become clearer, and that is the trigger for revisiting this validation item, not a fixed deadline or a specific percentage threshold.

Caveat for whoever picks this up next: because no concrete numbers were captured in that session, do not report a specific error margin, percentage gap, or match number for this comparison anywhere unless the user provides it again or a new comparison is run and logged with specifics.

13.4 Known, already-understood sources of expected divergence

- Bug 2 from Stage 7 (Section 11.4/11.7) — the live Google Sheet itself truncates its alliance-combined score curve at 500, silently discarding real probability mass for the roughly one-third of alliance scores that exceed 500 in the original competition's data. This app deliberately does NOT reproduce that truncation, so some divergence from the live sheet is not just expected but desired — this app's number is the more accurate one in that case.
- The Blue/Red cross-labeling question (Section 6.5) — this app builds each alliance's curve from its own three teammates directly, rather than the sheet's internal cross-referenced labeling. Mathematically equivalent but not a byte-for-byte implementation match, and remains a plausible (though still unconfirmed) partial explanation for any remaining percentage gap beyond what Bug 2 alone would explain.

13.5 What the next session should do

The user's stated plan (as of Stage 8) was to test the app against a different competition's data next and see what breaks. That happened during the Stage 9 session — the user reported “everything worked within expectation” apart from the Stage 9 bug — and has not yet been repeated with a fresh Google Sheet comparison against that or any other competition's data. Revisiting Section 13's bookmarked validation with a second competition's real numbers (this time logging specifics — see Section 19) is still open and is safe to do without the Stage 9 bug interfering, since a “New competition” import correctly isolates each event's data. Stages 10–13 did not touch this item at all — it remains exactly as open as it was in the v5 PDF.

14. Stage 9 — Bug Fix: New-Competition CSV Import Mixing With Old Competition's Data

14.1 Symptom reported

The user tested the app against a different competition's worth of data (Match Schedule + Raw Data CSVs from a second event) on top of an app instance that already had a first competition's data loaded from earlier testing. The user reported: “Everything works within expectation but a new minor bug was found. If I have the app filled with one competition's data and then I go to add a new competition's worth of data, it does not overwrite the existing data. This means that it is mixing old data with new data. When I upload a CSV it should replace the old data with the new data.” Beyond this one issue, the user confirmed the new competition's data otherwise worked as expected — no other bugs were reported from this second-competition test pass.

14.2 Root cause

Both `data_import.py::import_match_schedule_csv()` and `data_import.py::import_raw_data_csv()` already had, and had always had (since their Stage 4 design), a `replace_existing: bool` parameter specifically intended for exactly this “start fresh with a new competition” scenario. The bug was simply that `ui/app.py's _import()` method never used it — it always called both importers with the `merge/upsert` default, `replace_existing=False`.

This had two distinct, compounding effects, confirmed by direct reproduction (Section 14.4):

- `match_schedule` has `match_num` as its primary key, so a merge-mode re-import upserts by match number rather than duplicating. This is the CORRECT, desired behavior for updating the SAME competition later (e.g. importing official scores after the matches finish, without disturbing team assignments already imported — see Section 19.3/19.5's “partial upsert” design). But a second, different competition almost always reuses match numbers 1, 2, 3, ... from the start of its own schedule, so importing it in merge mode silently overwrote the first competition's match rows one-for-one by match number, rather than being kept separate or replacing the whole table — the two competitions' schedules ended up interleaved with no way to tell which match belonged to which event, and any of the first competition's matches numbered higher than the second competition's total match count were left behind, still mixed in.
- `manual_raw_data` has no natural key at all — just a plain auto-incrementing id. Merge-mode import doesn't (and structurally can't) upsert into it — it just appends every row from the new CSV on top of whatever rows were already there. Every raw-data import, even re-importing the exact same file twice, added a second full set of rows, silently doubling (and, across more imports, further compounding) every downstream chronological/statistical calculation that reads from `manual_raw_data`: Score Variance, Balls Scored, Raw Data Processing, and everything built on top of them (Defense Adjustment, Average Compatability, RDP, ROP, RPP, Match Odds).

14.3 Fix

`ui/app.py` gained a new `ImportModeDialog` class — a small modal `Toplevel` shown before every CSV import (both Match Schedule and Raw Data), asking the user directly: “Is this CSV more data for the competition already loaded, or the start of a different competition?” with three buttons:

- “Update current competition (keep existing data)” — sets `mode = “merge”`. Both importer functions are then called exactly as before Stage 9 (`replace_existing=False`). This preserves the original, correct behavior for the same-competition use case: adding official scores to a schedule already imported, re-importing a raw-data CSV with new or corrected rows, etc.

- “New competition (replace ALL existing data)” — sets mode = “replace”. Before either importer runs, the new `database.clear_active_competition_data()` function is called, which deletes every row from BOTH `match_schedule` AND `manual_raw_data` in a single operation — not just whichever of the two tables the CSV about to be imported populates. Both importer functions are then also called with `replace_existing=True` (a belt-and-suspenders match to the wipe that already happened, and correct behavior in its own right for that table).
- “Cancel” — aborts the import entirely; no dialog result, no file processing, no database changes.

Wiping BOTH tables together (rather than, say, having Match Schedule's import only clear `match_schedule` and Raw Data's import only clear `manual_raw_data`) was a deliberate design choice: the two supported CSVs are usually imported in “Match Schedule first, then Raw Data” order, but nothing enforces that order, and a partial per-table clear would still let the OTHER table's old-competition data linger and mix with the new competition's data if the user happens to import the two CSVs in the less common order, or imports only one of the two CSVs for the new event before the other. A full two-table wipe on “New competition” makes the fix correct regardless of import order.

The import-complete message box (shown after every successful import) now prepends “Cleared previous competition's data.” to its usual summary whenever replace mode ran, so it's visually obvious after the fact which mode was used for that import.

14.4 Validation results

Reproduced the bug first, then confirmed the fix, both headlessly against the real `Untitled_spreadsheet_-_Match_Schedule.csv` and `Untitled_spreadsheet_-_Raw_Data.csv` files, by calling the importer functions directly (bypassing the UI) with the exact `replace_existing` values each `ImportModeDialog` choice now produces:

- Bug reproduction (old behavior — both imports always merge-mode, matching pre-Stage-9 `ui/app.py`): importing the Match Schedule CSV twice with `replace_existing=False` left `match_schedule` at 110 rows (upserted correctly by primary key — not the buggy part). But importing the Raw Data CSV twice with `replace_existing=False` took `manual_raw_data` from 625 rows to 1250 rows — an exact, reproducible confirmation of the reported “mixing old data with new data” symptom, isolated specifically to the table with no natural key.
- Fix confirmation (“New competition” path): calling `database.clear_active_competition_data()`, then `import_match_schedule_csv(path, replace_existing=True)` and `import_raw_data_csv(path, replace_existing=True)` — a second time, on top of an already-loaded first “competition” (the same CSVs, standing in for a different event) — left `match_schedule` at exactly 110 rows and `manual_raw_data` at exactly 625 rows: no duplication, no leftover rows from the earlier load. `calculations.pipeline.build_all(my_team_num=1796)` was then run against this freshly-replaced data and produced the expected 75-team roster with 625 raw-data rows, matching the original single-import baseline (Section 9.5) exactly.
- Fix confirmation (“Update current competition” path, to confirm the legitimate merge use case wasn't broken): importing the Match Schedule CSV twice in a row with `replace_existing=False` (simulating a user re-importing an updated schedule for the SAME event, e.g. to add scores) correctly left `match_schedule` at 110 rows both times — confirming the primary-key upsert behavior this mode depends on is intact and unaffected by the Stage 9 changes.
- Every `.py` file in the project was re-syntax-checked with `ast.parse()` after the Stage 9 edits and all parsed cleanly.

14.5 What did NOT change

Neither `import_match_schedule_csv()` nor `import_raw_data_csv()` had their own internal logic changed at all in Stage 9 — both functions' `replace_existing` parameter, and everything each function does with it internally (the `DELETE FROM ...` statement each already ran at its own top when `replace_existing` was passed as `True`), existed unchanged since Stage 4/5. Stage 9 is purely a UI wiring fix: the two importer functions are now finally called with the `replace_existing` value the user actually intends, via the new dialog, instead of always being called with `False`. `calculations/`, `database.py`'s `SCHEMA`, and every calculation file are completely untouched. `database.clear_active_competition_data()` deliberately does NOT touch `raw_scouting`, `bps_samples` (both are legacy/unused since Stage 4 — see Section 19.1) or `team_roster` (a small local naming table that isn't competition-specific and has never been populated by the current two-CSV import path anyway).

14.6 Code changes summary (Stage 9)

database.py — added `clear_active_competition_data()`, which runs `DELETE FROM manual_raw_data` and `DELETE FROM match_schedule` inside a single connection/commit. `ui/app.py` — added the `ImportModeDialog` class (`tk.Toplevel` subclass with three buttons; `.show()` blocks via `wait_window()` and returns “merge” / “replace” / `None`); `_import()` now shows this dialog immediately after the file picker returns a path and before either importer function is called, calls `clear_active_competition_data()` when the user picked “New competition”, and passes `replace_existing=(mode == “replace”)` into both `data_import.import_raw_data_csv()` and `data_import.import_match_schedule_csv()`; both post-import message-box branches now prepend a “Cleared previous competition’s data.” line when replace mode ran. `README.md` — new “Stage 9” section documenting the bug, root cause, fix, and validation results (mirroring this section).

15. Stage 10 — Team Lookup: Score Trend Chart

15.1 Request

The user asked, on the Team Lookup tab, for a graph showing the score variance of the selected team over time.

15.2 Implementation

A new “Score Trend” chart was added to `ui/team_lookup_tab.py`, placed at the top of the tab’s body — below the “Team {n}” header and above the existing stat panels — so it’s the first thing visible after looking up a team. It plots `self.data.score_variance[team].values`: the exact per-team, chronologically ordered, ≤ 15 -match list produced by `calculations/score_variance.py::build_score_variance()` (see Section 6.2), which is the same list the (at the time) still-present “Score Variance” text panel computed its Average/Std Dev/Skew/RRP figures from.

The chart is a line plot with markers (one point per scouted match, in scouting order — not the real FRC match number, since that per-team value list has no match number attached to each entry, matching the source sheet’s own convention of laying a team’s history across columns B:P by chronological appearance with no match-number header), plus a dashed horizontal line at the team’s average score, labeled with its value, and a light grid. If a team has no `score_variance` row (e.g. an unrecognized team number), the chart renders a centered “No scouted score history for this team.” placeholder instead of erroring.

Implementation is `matplotlib`’s `Tkinter` (`TkAgg`) backend, embedded directly via `matplotlib.backends.backend_tkagg.FigureCanvasTkAgg` inside a new `_draw_score_chart()` method, called from the tab’s existing `_render()` flow. Because `_render()` already destroys and rebuilds the tab’s entire body on every lookup (as it did before this stage), the chart widget is naturally destroyed and rebuilt along with everything else — no separate chart-specific cleanup/lifecycle code was needed. `matplotlib.figure.Figure` is used directly (not the `pyplot` state-machine interface), which is the recommended pattern for embedding in a long-lived `Tkinter` app to avoid the memory growth `pyplot`’s global figure registry would otherwise cause. `matplotlib>=3.8` was added to `requirements.txt` — see Section 5 (as of Stage 13, this is the project’s only pip dependency, and its two import statements now live inside `_draw_score_chart()` rather than at module level — see Section 18).

15.3 Validation results

Ran the full real-data import pipeline (Match Schedule + Raw Data CSVs) fresh: 625 Raw Data rows imported, 29 skipped, 34 `schedule_conflicts`, 1 `likely_typo_corrections`, and a resulting 75-team roster — exactly matching the established Section 9.5 baseline, confirming no regression. Pulled team 1796’s real Score Variance history (8 scouted matches: 302.72, 241.77, 245.68, 289.47, 304.99, 183.94, 278.78, 50.51; average ≈ 237.23) and rendered the exact chart-drawing logic headlessly via `matplotlib`’s non-interactive `Agg` backend to a PNG file, confirming it built and saved without error and visually matched the expected shape (a line connecting the 8 points with a dashed average line at ~ 237). Separately confirmed the empty-history branch (an unrecognized team, returning `sv_row=None`) renders its placeholder text and does not raise an exception. Every `.py` file in the project was re-syntax-checked with `ast.parse()` and all parsed cleanly.

Unlike Stages 8 and 9, this stage’s change WAS subsequently confirmed by the user on their own real machine (not just headlessly in the build environment): the user’s next message reported “The graph looks good” before moving on to request the Stage 11 cleanup.

15.4 What did NOT change

No file under calculations/ was touched. `build_score_variance()` and every other calculation in the pipeline run exactly as before; this stage only added a new way to visualize data that already existed. `requirements.txt` gained one new line (`matplotlib`); no other dependency changed at this stage (`numpy`, `scipy`, and `pandas` were still present in `requirements.txt` at the end of Stage 10 — they weren't removed until Stage 13; see Section 18).

15.5 Code changes summary (Stage 10)

`ui/team_lookup_tab.py` — added `matplotlib` imports (`Figure`, `FigureCanvasTkAgg`); module docstring updated; `_render()` now builds a “Score Trend” `ttk.LabelFrame` immediately after the team header and calls the new `_draw_score_chart(parent, sv_row)` method, which builds the line chart (or the empty-state placeholder) described in 15.2. `requirements.txt` — added `matplotlib>=3.8` with an inline comment explaining why (BSD-licensed, no network calls, `TkAgg` backend draws into the existing window). `README.md` — new “Stage 10” section documenting the same information as this section. Delivered as `frc_scouting_app_12.zip`.

16. Stage 11 — Team Lookup: Panel Cleanup

16.1 Symptom reported

Immediately after confirming Stage 10's chart looked good on their real machine, the user reported they could no longer scroll on the Team Lookup tab to see the rest of the information — the tab (now taller with the chart added on top of all 7 original stat panels) no longer fit on screen, and there was no scrolling mechanism to reach the panels below the fold.

16.2 Panels removed and why

Rather than build a scrolling feature immediately, the user asked to first see whether trimming the tab's content down was enough on its own. Three of the tab's seven stat panels were removed, all purely in `ui/team_lookup_tab.py`:

- Score Variance — removed as redundant: its Average/Std Dev/Skew/RRP figures are the exact same data the Stage 10 chart now shows visually (including the average, via the chart's dashed line).
- Skew Adjusted Spread — removed at the user's explicit statement that this wasn't something users referenced.
- Defense — removed as redundant: while its “Opponent-suppression RRP” (from Defense Adjustment) is arguably distinct information, the tab's existing Overall Power panel already surfaces the practical, at-a-glance takeaway derived from this same data — the “Weak to Defense?” flag (`rpp.defense_susceptible`) — so the separate panel was judged redundant for this tab's purposes.

16.3 Layout after the change

The remaining four panels were reflowed from the original 5-row/7-panel grid down to a 3-row/4-panel grid, with no gaps left where the removed panels used to sit: row 0 is Auto Balls Scored and Teleop Balls Scored side by side; row 1 is Fitted Score Distribution and Average Compatability side by side; row 2 is Overall Power, spanning both columns. A code comment was left in place of the removed panels explaining what was removed, why, and that the underlying data is still fully available on `self.data` if a future stage wants to bring any of it back.

16.4 Validation / outcome

Every `.py` file was re-syntax-checked with `ast.parse()` and all parsed cleanly; the resulting grid structure was reviewed by inspection against the same layout pattern already used elsewhere in the file (`ttk.LabelFrame` in a 2-column `ttk.Frame` grid). The real test, however, was the user's own machine: after receiving the updated code, the user reported “Looks good. I don't think a scroll feature is needed since cleaning up the tab fixed the issue. The team lookup tab looks great now.” This is a direct, real-machine confirmation that the panel cleanup alone solved the fit problem — no scrolling feature exists anywhere in the project as of this writing, and none is currently planned, since the user's own stated goal (“see if this cleans it up enough to not need a scroll feature”) was met.

16.5 What did NOT change

No file under calculations/ was touched, and no calculation was disabled or skipped. `self.data.score_variance`, `self.data.skew_adjusted_spreads`, `self.data.defense_calculations`, and `self.data.defense_adjustment` are all still fully populated by `calculations/pipeline.py::build_all()` on every run — only their corresponding Tkinter display panels in `ui/team_lookup_tab.py` were removed. Restoring any of the three removed panels in a future session is a matter of re-adding the relevant `ttk.LabelFrame + stat_row()` calls (the exact code was removed, not commented out, so a future

session would need to either rewrite it from these field names or pull it from an earlier zip such as `frc_scouting_app_12.zip`, the last version that still had all seven panels).

16.6 Code changes summary (Stage 11)

`ui/team_lookup_tab.py` — module docstring updated; the “Score Variance”, “Skew Adjusted Spread”, and “Defense” `ttk.LabelFrame` panels and their associated `stat_row()` calls were removed from `_render()`; the remaining four panels' grid row/column assignments were updated to the new 3-row layout described in 16.3; a comment documenting what was removed and why was left in their place. No other file was touched. `README.md` — new “Stage 11” section documenting the same information as this section. Delivered as `frc_scouting_app_13.zip`.

17. Stage 12 — Match Lookup: Score Distribution Chart

17.1 Request

The user asked, below the existing information on the Match Lookup tab, for a graph showing the skewed distribution for the red alliance and the blue alliance, on the same plot, so the overlap and differences between the two curves would be visually obvious.

17.2 Implementation

A new “Score Distribution” chart was added to `ui/match_lookup_tab.py`, packed below the existing two-column red/blue alliance panels (and below the win-odds-unavailable error label, if one is shown). It overlays both alliances' combined-score curves — red in one color, blue in another, each with a semi-transparent fill — on a single set of axes, with a legend and axis labels.

Data source: each alliance's `AllianceDistribution.empirical_curve`, produced by `calculations/alliance_generator.py::build_alliance_distribution()` (Section 6.4) — the exact same curve `calculations/match_odds.py::compute_match_odds()` already reads to compute the win probability and expected score shown in the panels above the chart. `_render()` was restructured so `red_dist` and `blue_dist` (previously local variables inside the try block that computes odds) are initialized to `None` beforehand and assigned inside that same try block, so they remain available afterward for the chart to reuse — the chart does not recompute anything; it plots the same distribution objects the odds calculation already built for that render.

17.3 Why the raw curve needed display-only smoothing

`empirical_curve` is built via an exact-match lookup onto the 1500-point integer axis described in Section 6.6 (`X_AXIS_MIN..ALLIANCE_X_AXIS_MAX`), mirroring the source sheet's own `VLOOKUP` exact-match behavior: an integer score with no exact combined sample lands on density exactly 0, with no smoothing or interpolation between neighboring scores. This is a real, confirmed property of the algorithm (not a bug) — direct inspection of a real match's curve found regions with a genuine, repeating comb-like oscillation (e.g. one real blue alliance curve alternated between roughly 0.98, 0.35, and 0.0 density across a 3-point period around score ~600–650). Plotted raw, this renders as a dense forest of spikes rather than a readable curve, making it very difficult to visually judge two alliances' overlap — directly undermining the point of the chart the user asked for.

`_draw_alliance_chart()`'s solution, applied strictly FOR DISPLAY PURPOSES ONLY inside `ui/match_lookup_tab.py`: each curve is passed through a symmetric moving average (window = 25 integer-score points) via a small nested `smoothed()` helper (implemented with a running prefix sum for efficiency), then only every 6th resulting point is plotted (stride = 6) to remove residual small-amplitude ripple left over from the averaging. Each smoothed curve is then independently rescaled so its own peak is 1.0, since the chart's purpose is to compare relative shape and overlap between the two alliances, not absolute density (which is already presented numerically, unsmoothed, in the expected-score/win-probability panels above the chart). Critically, none of this smoothing touches `red_dist`, `blue_dist`, or any calculation anywhere — `calculations/alliance_generator.py` and `calculations/match_odds.py` are both completely unmodified, and every number in the panels above the chart is still computed from the raw, unsmoothed `empirical_curve` exactly as it was before this stage. If a future session ever needs the exact raw curve for a non-visual purpose, `red_dist.empirical_curve` and `blue_dist.empirical_curve` remain unchanged and directly accessible.

If `red_dist` or `blue_dist` is `None` (i.e. the existing win-odds calculation above the chart could not run — for example a missing fitted distribution for one of the six teams in the match), the chart renders a centered “Score distribution

unavailable for this match.” placeholder instead of erroring, mirroring the empty-state pattern already established by Stage 10’s Team Lookup chart.

17.4 Validation results

The exact smoothing/trimming logic used by `_draw_alliance_chart()` was extracted and run against all 110 real matches in the user’s Match Schedule CSV: all 110 produced a valid, full-length (1500-point) smoothed curve for both alliances with zero exceptions. Three real matches spanning the win-probability range were then rendered headlessly, using matplotlib’s non-interactive Agg backend, to PNG files for visual inspection:

- Match 1 (18.9% red / 81.1% blue, a lopsided match): the two smoothed curves appeared clearly separated, blue’s shifted well to the right of red’s, with only modest overlap in the middle — visually consistent with the strongly blue-favored win probability.
- Match 15 (50.9% red / 49.1% blue, a near-coinflip): the two curves appeared almost fully overlapping across their whole range — visually consistent with a near-even match.
- Match 50 (75.7% red / 24.3% blue): rendered without error, following the same red-favored pattern as match 1 but mirrored.

This is exactly the intended behavior — the degree of visual overlap tracks the actual win probability — confirming the chart serves the purpose the user asked for. Every .py file in the project was re-syntax-checked with `ast.parse()` and all parsed cleanly.

UNLIKE Stages 10 and 11, this chart has NOT yet been confirmed by the user on their own real machine as of this writing — all validation above was performed headlessly in the build environment (Agg backend PNG renders and the 110-match logic stress test), the same limitation Stages 8 and 9 had before Team Lookup’s charts were confirmed working. This remains open as of Stage 13 as well (Stage 13 did not attempt to close this item — it also had no display available; see Section 18.6) and is flagged as a top-priority next step in Section 19.

17.5 What did NOT change

`calculations/alliance_generator.py` and `calculations/match_odds.py` are both completely unmodified by this stage — `build_alliance_distribution()` and `compute_match_odds()` are called exactly as they were as of Stage 7, and every number in the Match Lookup panels above the new chart (each team’s Score Variance avg/RRP, the actual recorded score, the expected score, and the win probability) is computed identically to before this stage. No new dependency was added — this stage reuses the matplotlib/TkAgg embedding pattern Stage 10 already established (Figure + `FigureCanvasTkAgg`, not the `pyplot` interface).

17.6 Code changes summary (Stage 12)

`ui/match_lookup_tab.py` — added matplotlib imports (`Figure`, `FigureCanvasTkAgg`); module docstring updated; `_render()` now initializes `red_dist = blue_dist = None` before the odds-calculation try block (instead of leaving them scoped only inside it), and after the existing odds/odds_error handling, builds a new “Score Distribution” `ttk.LabelFrame` and calls the new `_draw_alliance_chart(parent, red_dist, blue_dist)` method, which builds the smoothed overlay chart (or the empty-state placeholder) described in 17.2–17.3. No other file was touched — `alliance_generator.py`, `match_odds.py`, `database.py`, and `data_import.py` are all unaffected. `README.md` — new “Stage 12” section documenting the same information as this section. Delivered as `frc_scouting_app_14.zip`.

18. Stage 13 — Startup Performance Optimization

18.1 Request

The user reported the app takes a while to open on initial startup and asked whether startup time could be optimized further — explicitly without affecting any GUI behavior or backend calculation results.

18.2 Investigation method

Profiled the app’s cold-start import chain using Python’s “-X importtime” flag, and benchmarked `calculations/pipeline.py::build_all()` end-to-end against the same real Match Schedule / Raw Data CSVs used throughout this document (Section 19.6), both before and after each candidate change, comparing full pipeline output for exact equivalence. No GUI framework was available in the build environment used for this session — no display, and `tkinter`

itself is not installed there at all (`import tkinter` fails) — the same standing limitation Stage 8 and Stage 9 noted; see 18.6.

18.3 Root cause found — `scipy.stats.norm.pdf`

Grepping the entire project confirmed `numpy` and `pandas` — both listed in `requirements.txt` since the original build — are never imported anywhere in the codebase. `scipy` was imported for exactly one thing:

`calculations/distributions.py::build_skew_distributions()` calls `scipy.stats.norm.pdf()` once per team per whole-number `x` from 1 to `X_AXIS_MAX` (500) — roughly 37,500 calls for the user's real 75-team roster.

Two compounding costs were measured: (1) importing `scipy` at all (which pulls in `numpy`) costs real time by itself before a single call is made; (2) `scipy.stats.norm.pdf()` routes each call through `scipy`'s generic `rv_continuous` distribution-object dispatch machinery, benchmarked at roughly 125x slower per call than evaluating the same closed-form normal-PDF formula directly with the standard-library `math` module (32.6 microseconds/call vs. 0.26 microseconds/call, measured over 20,000 calls of each in the build environment; results also verified numerically identical to within $\sim 7e-18$ across 1,000 random parameter combinations).

18.4 Fix

`calculations/distributions.py` — replaced “`from scipy.stats import norm`” and its call site (“`norm.pdf(tx, loc=mean_adj, scale=stdev_adj)`”) with a new private `_normal_pdf(x, loc, scale)` function: a ~ 10 -line closed-form implementation using `math.exp/math.sqrt`, plus a module-level `_SQRT_2PI` constant. `numpy`, `scipy`, and `pandas` were removed from `requirements.txt` entirely — `numpy/pandas` were dead weight (never imported anywhere), and `scipy` was no longer needed anywhere in the codebase once its one call site was gone.

`matplotlib`'s imports in `ui/match_lookup_tab.py` and `ui/team_lookup_tab.py` were moved from module level down into the two chart-drawing methods that actually use them (`_draw_alliance_chart()` and `_draw_score_chart()`, respectively) — so a fresh install with no CSV imported yet (empty database, nothing to chart) no longer pays `matplotlib`'s import cost at startup. Once a competition's data is loaded, `matplotlib` is still imported at startup exactly as before this stage, since both tabs' `_render()` draws a chart immediately on the first refresh.

18.5 Validation results

Ran `calculations.pipeline.build_all()` against the exact same real Match Schedule/Raw Data CSVs used throughout this document (Section 19.6), both before (original `scipy`-based `distributions.py`) and after (the new `_normal_pdf()` function), and compared every value the change could plausibly affect:

- `skew_distribution_curves` (the direct output of the changed function): maximum absolute difference across every team/`x`-value pair was $1.1368683772161603e-13$ — floating-point noise, not a real difference.
- `normalized_distributions'` `normalized` curves, `mode_x`, and `upper_cutoff`: maximum absolute difference $4.440892098500626e-16$; `mode_x` and `upper_cutoff` were bit-for-bit identical for every team.
- `pick_list` and `rpp` (the final, most downstream outputs any of this feeds): bit-for-bit identical (a structural Python `==` comparison returned `True`) before vs. after.

Timing, measured against that same real data: `calculations.pipeline.build_all()` alone dropped from 0.63s to 0.02s ($\sim 30\times$). The combined cost of “import `calculations.pipeline`, initialize the database, and run `build_all()` once” — i.e. everything Stage 13 could affect, excluding `tkinter/matplotlib`, which the build environment couldn't exercise without a display — dropped from ≈ 1.30 s to ≈ 0.04 s. Every `.py` file in the project was re-compiled with `py_compile` after the edits and all compiled cleanly.

18.6 Important caveat — not yet confirmed on a real machine

Unlike Stages 10 and 11 (and like Stage 12, still open — see Section 17.4 and Section 19's top-priority item), none of Stage 13's changes have been visually confirmed in an actual running Tkinter window. The build environment used for this session has no display and does not have `tkinter` installed at all (`import tkinter` fails there), so the Match Lookup / Team Lookup charts could not be rendered end-to-end the way Stage 10 and Stage 12 partially managed with the Agg backend — this session did not attempt an Agg-backend PNG render either, since the change under test was import timing and numeric output, not chart appearance, and neither chart-drawing method's visual output was touched. The lazy-import change is a pure code-motion (the exact same import statements, moved from module scope to function scope) and carries very low risk, but it has not been executed inside a real `tk.Notebook` the way it will be the first time the user runs the app again.

Confirm on the user's real machine, ideally in the same session: (1) the app still opens and both charts still draw correctly the first time each tab is visited (Match Lookup's Score Distribution chart and Team Lookup's Score Trend chart should look exactly as they did before this stage, since neither chart's drawing code changed — only when its imports run); and (2) the app's cold start is subjectively/measurably faster than before, ideally timed with a stopwatch from launching python main.py to the window becoming responsive.

18.7 What did NOT change

No calculation's inputs, formula, or output changed — see the bit-for-bit / floating-point-noise comparison in 18.5. No file under ui/ had any widget, layout, event binding, or visible behavior changed — only the location of two import statements moved from module scope to function scope in each of two files, with no change to what gets imported or when a chart is actually drawn relative to user action. database.py, data_import.py, and every file in calculations/ other than distributions.py are untouched.

18.8 A further, not-yet-implemented optimization worth knowing about

While profiling, it became clear that ui/app.py's ScoutingApp.recalculate() (called once during `__init__`, before the window's mainloop starts) eagerly calls `.refresh()` on all four tabs — including Team Lookup, Pick List, and Rankings, none of which is the tab shown first — meaning, for example, Team Lookup's chart is fully built (and matplotlib's import cost paid, if not already paid by Match Lookup) at startup even though the user cannot see it yet. Deferring non-visible tabs' `refresh()` until the user actually clicks on them (e.g. via a `<<NotebookTabChanged>>` binding, refreshing lazily and caching a “last refreshed for this self.data” flag per tab) would very likely shave additional real startup time off a populated database on top of Stage 13's fixes.

This was deliberately NOT implemented in this session, specifically because it changes the timing of GUI construction/event flow in a way that could not be verified without a real display, and the risk of subtly breaking tab-switch behavior outweighed the upside without being able to test it. Flagged here as a good candidate for the next session that does have GUI access — see also Section 19.

18.9 Code changes summary (Stage 13)

calculations/distributions.py — removed “from scipy.stats import norm”; added “import math”, a module-level `_SQRT_2PI` constant, and a private `_normal_pdf(x, loc, scale)` function; its one call site (inside `build_skew_distributions()`) now calls `_normal_pdf(tx, mean_adj, stdev_adj)` instead of `float(norm.pdf(tx, loc=mean_adj, scale=stdev_adj))`. requirements.txt — removed numpy, scipy, and pandas entirely; kept matplotlib with an updated comment explaining both why scipy/numpy/pandas were removed and that matplotlib is now imported lazily. ui/match_lookup_tab.py — “from matplotlib.figure import Figure” / “from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg” moved from module level into `_draw_alliance_chart()`. ui/team_lookup_tab.py — the same two imports moved from module level into `_draw_score_chart()`. No other file was touched. Delivered as frc_scouting_app_15.zip.

19. Suggested Next Steps (in priority order)

- **Top priority** — confirm, on the user's real machine, the two still-open items from the two most recent sessions together: (1) Stage 12's Match Lookup “Score Distribution” chart (Section 17) — still only validated headlessly (Agg-backend PNG renders, a 110-match logic stress test); and (2) Stage 13's startup-performance changes (Section 18) — the lazy matplotlib imports and the scipy-free curve-fit have only been validated by numeric diffing and import-timing in a headless, display-less build environment, never inside a real Tkinter window. Confirm: the app opens noticeably faster than before; both Match Lookup's and Team Lookup's charts still render correctly and look exactly as they did before Stage 13 (they should — neither chart's drawing code changed); the Score Distribution chart's layout inside its `ttk.LabelFrame` doesn't crowd the alliance panels above it; and the legend/labels are legible at the window's actual default size.
- Use a second (or the already-tested different) competition's data to make progress on Section 13's bookmarked Google Sheet validation. If that event also has a live Google Sheet with real Match Odds to compare against, run the same kind of spot-check the user did in the Stage 8 session, but this time capture specifics (which match, both systems' win percentages, both systems' expected scores if the sheet exposes them) so a future session has real numbers to root-cause against instead of a qualitative impression. This item is unchanged by Stages 10–13 and remains exactly as open as it was in the v5 PDF.

- Given the Stage 7 finding that individual-robot 95th-percentile cutoffs run as high as ~589 in the original competition's data (Section 6.6), re-check that figure against whatever range the newly-tested competition's data shows — a different event could easily push `X_AXIS_MAX` or `ALLIANCE_X_AXIS_MAX` further, and it's now safe to check this on genuinely separate, non-mixed competition data thanks to the Stage 9 fix.
- Keep an eye on real-world Pick List / Rankings quality across events now that Stage 6 (plain Score Variance) is live in production use, and compare informally against what the live Google Sheet's Pick List would have suggested, if practical. The revert path in Section 10.4 is there if this doesn't pan out.
- Consider deferring non-visible tabs' `refresh()` until first shown (see Section 18.8) — Team Lookup, Pick List, and Rankings are all fully rebuilt at startup before the user can see them, which likely costs additional real startup time on top of what Stage 13 already fixed, especially once a competition's data is loaded (matplotlib's import, chart draws, and DB queries all happen for tabs the user hasn't clicked on yet). Not implemented in Stage 13 because it changes GUI construction/event-flow timing that couldn't be verified without a real display.
- Update README.md with any future stage's write-up if it's ever found to be out of sync with this PDF — as of this handoff the two are in sync through Stage 13 (README.md's Stage 10, 11, 12, and 13 sections were written alongside each stage's code changes).
- If the user encounters more Match Schedule typos like the match-47/team-325 case (Section 9.4) in a new competition's data, the existing `likely_typo_corrections` reporting should surface them automatically on import — but it's worth periodically reviewing the `schedule_conflicts` list too.
- Optional polish, updated for what Stages 10–12 already delivered: plotting a team's fitted score curve or an alliance's combined distribution inline in the Team Lookup / Match Lookup tabs — the item suggested in the v5 PDF — is now DONE (Section 15's Score Trend chart and Section 17's Score Distribution chart). Remaining/new optional ideas: a results/history view comparing Match Odds predictions to actual outcomes over a season (Section 11.5's 110-match backtest logic would be a good starting point, and Stage 12's per-match smoothing helper could be reused for any per-match chart such a view might want); a “current competition” label somewhere in the UI (e.g. the status bar) so it's visually obvious which event's data is loaded, now that switching between competitions is a supported, one-dialog operation rather than an error-prone manual process; optionally marking each alliance's actual recorded score and/or expected score as a vertical line on Stage 12's Match Lookup chart, now that the chart infrastructure exists, if the user finds that useful once they've tried the current version; and matplotlib's built-in `NavigationToolbar2Tk` (zoom/pan/save) could be added to either chart if the user wants to interact with them beyond the default static view.
- If the “Super Scouting” sheet (§2.4) ever resurfaces, its formulas could be added to fill in Team Lookup's team-name / defense-notes / climb-confidence fields.
- If official alliance scores for additional matches become available for an already-loaded competition, re-import an updated Match Schedule CSV using “Update current competition” mode — matches that already had a score keep using the exact apportioned formula, and any match that gains a score for the first time will automatically switch over from the raw-score fallback (Section 19.4) to the exact formula on the next Recalculate. Do NOT use “New competition” mode for this, since that will clear all previously-imported raw-data for the event.

20. Data Import Redesign (Stage 4 detail, historical reference)

This section is preserved from the original Stage 4 handoff PDF as a historical/reference record of the base two-CSV import design. It describes the import model as it stood at the end of Stage 4, before the Stage 5 fixes in Section 9 and before the Stage 9 import-mode fix in Section 14. Stages 10, 11, 12, and 13 did not touch the import system at all. Where Stage 5 or Stage 9 changed the behavior described here, those sections are the current, correct description — this section is kept for the parts that are still accurate (the CSV column formats, the two-row header detection, the recommended import order) and for context on why the two-CSV model looks the way it does.

20.1 What was removed, and why

The original design had three CSV imports feeding three input tables: `raw_scouting` (granular, stopwatch-timed scouting entries), `bps_samples` (per-team ball-per-second calibration samples), and `match_schedule`. The user does not scout with a stopwatch — they pre-calculate each robot's full score breakdown themselves — so the `raw_scouting/bps_samples` path was never going to be used. At the user's explicit request, both were removed from the Data menu. Neither removal touched the underlying calculation code — `calculations/raw_processing.py` still contains `team_bps_averages()`,

`_augment_unfiltered_rows()`, and `build_raw_data()` exactly as before, clearly marked in the module docstring as legacy/unused.

20.2 The active Raw Data CSV format

“Import Raw Data CSV (Team + Score Breakdown)...” is the sole source of Raw Data for the entire pipeline. One row per scouted robot per match. Implemented in `data_import.py::import_raw_data_csv()`. Recognized columns (header matching is case/space/underscore-insensitive): Team Number (required), Match Number (required), Alliance/Position (optional, fallback-only as of Stage 5 — see Section 9.1), Auto Shooting/Auto Balls, Auto Climb/Auto L1, Teleop Shooting/Teleop Balls, Teleop L1/L2/L3, Score/Total Score (derived from the breakdown if blank, defaulting to 2 if that sum is exactly 0), Notes (optional).

Header row detection (`data_import.py::_detect_raw_data_header()`) supports two formats automatically: a single-row header, or the two-row grouped header the user's actual export uses. Team numbers with thousands-separator commas (e.g. “1,796”) are handled automatically. Blank/placeholder rows are silently skipped and counted, not treated as errors.

20.3 The active Match Schedule CSV format

“Import Match Schedule CSV...” is listed FIRST in the Data menu because it is the recommended import order. Implemented in `data_import.py::import_match_schedule_csv()`. 7 required columns (Match Number — header may be blank; Red 1/2/3; Blue 1/2/3), 2 optional trailing columns (Red Score, Blue Score — if a CSV omits these entirely, existing values are left alone rather than nulled out, when in “Update current competition” / merge mode). Uses a per-column partial upsert in merge mode, so a scores-only re-import never wipes out team assignments already established, and vice versa — this partial-upsert behavior is exactly what Stage 9's “Update current competition” mode preserves unchanged; it is deliberately bypassed (in favor of a full wipe of both tables) only when the user explicitly picks “New competition” — see Section 14.3.

20.4 Bug found and fixed in Stage 4: Adjusted Score zero-collapse

`build_raw_data_processing()` apportions each robot's raw scouted score against its alliance's OFFICIAL recorded match score. Originally, if that official score was still NULL, the apportionment formula silently coerced the missing score to 0.0, zeroing the Adjusted Score for every robot in that match. Fix (still in place, unchanged since): for any match still missing an official `red_score/blue_score`, `build_raw_data_processing()` falls back to using each robot's raw scouted score AS the adjusted score directly (apportionment ratio of 1.0). The instant a real official score is imported for that match, the exact original apportioned formula automatically takes over on the next recalculation.

20.5 Final Data menu

(menu items unchanged through Stage 13; import BEHAVIOR changed in Stage 9 — see Section 14): Data → Import Match Schedule CSV... / Import Raw Data CSV (Team + Score Breakdown)... / —separator— / Set My Team Number... / Recalculate Now. As of Stage 9, selecting either import command opens the `ImportModeDialog` first (Section 14.3) before the file's contents are actually written to the database.

20.6 Real-data validation performed in Stage 4

The user's actual Match Schedule CSV (110 matches) and Raw Data CSV (654 data rows) were imported through the production import functions for the first time: Match Schedule 110/110 rows imported successfully; Raw Data 625 rows imported, 29 correctly auto-skipped, 0 unresolved-alliance rows (at that time — Stage 5 later found and fixed real correctness issues in how those rows were being resolved). Full pipeline ran end-to-end with no errors: 75 teams, Goal Ally RPP ≈ 0.773 , 74-entry Pick List. Match Odds sanity check for match 1 produced a blue win probability of $\approx 61.4\%$ with expected scores of roughly 266 (red) / 300 (blue); the match's actual recorded final score was red 263 / blue 642 — the model's direction was correct even though the margin was badly underestimated, which is the exact symptom that led to the Stage 7 investigation (Section 11).

21. How to Resume This Project in a Fresh Session

Provide the following:

- This PDF (project context and every design decision made, through Stage 13).

- `Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv` — the ground-truth formula export, required for verifying any existing calculation or adding a new one. (Not required to re-verify Stage 13 specifically — that stage's validation was a before/after numeric diff against the app's own prior output, not against this CSV, since no formula or calculation output changed. Still required for any other calculation work.)
- The current `frc_scouting_app/` project folder (or its `.zip` — `frc_scouting_app_15.zip` as of the end of the Stage 13 session, or a newer-numbered zip if one has been shared since). Every `.py` file's docstring is the authoritative spec for what it implements.
- The user's real `Untitled_spreadsheet_-_Raw_Data.csv` and `Untitled_spreadsheet_-_Match_Schedule.csv` files (or the newer competition's CSVs tested during the Stage 9 session, if picking up Section 13's Google Sheet validation item) — these are the exact files used for every validation number in Sections 9, 10.5, 11.5, 14.4, 15.3, 17.4, and 18.5.

With those, a fresh session can:

- Re-run the real-data pipeline test to confirm nothing has regressed: re-import the two CSVs into a fresh `data/scouting.db` via `data_import.import_match_schedule_csv()` then `data_import.import_raw_data_csv()`, then call `calculations.pipeline.build_all(my_team_num=1796)`. Expect: 625 imported, 29 skipped_missing_ids, 0 skipped_ambiguous_alliance, 34 schedule_conflicts, 1 likely_typo_corrections (match 47, 325→3256), and a resulting 75-team roster with a 74-entry Pick List — this is the ORIGINAL competition's data; a different competition's numbers will differ and that's expected. None of Stages 10–13 changes these figures: Stages 10–12 touched no calculation code, and while Stage 13 changed `calculations/distributions.py`'s implementation, it was verified to produce bit-for-bit / floating-point-noise-identical output (Section 18.5).
- Look up any cell's real formula by filtering the `FORMULA_EXPORT` CSV on the Sheet column, exactly as was done throughout Sections 9 and 11's investigations.
- Re-validate the Stage 7 fix specifically: for each of the 110 matches in `match_schedule` with a non-null `red_score/blue_score`, build each alliance's distribution and compare expected scores against real recorded scores. Expect a mean absolute error around 100 points and a win-direction accuracy around 69% (76/110), matching Section 11.5.
- Confirm the Stage 8 UI changes are present: Match Lookup and Team Lookup should show type-in Entry fields with a “Go” button rather than dropdowns, and Rankings should show a “Sort by” dropdown above the table.
- Confirm the Stage 9 fix is present: choosing either “Import Match Schedule CSV...” or “Import Raw Data CSV...” from the Data menu should first open a small dialog asking “Update current competition” vs. “New competition” vs. “Cancel”, before any file is actually read into the database. Re-run the Section 14.4 headless reproduction/confirmation test if you want to re-verify the fix programmatically without the GUI.
- Confirm the Stage 10 chart is present: Team Lookup should show a “Score Trend” line chart at the top of the tab, immediately below the “Team {n}” header, plotting the selected team's chronological scouted-score history with a dashed average line.
- Confirm the Stage 11 cleanup is present: below that chart, Team Lookup should show exactly four stat panels — Auto Balls Scored, Teleop Balls Scored, Fitted Score Distribution, Average Compatability, and Overall Power (Overall Power spanning both columns) — with no separate Score Variance, Skew Adjusted Spread, or Defense panels.
- Confirm the Stage 12 chart is present: Match Lookup should show a “Score Distribution” chart below the red/blue alliance panels, overlaying both alliances' combined-score curves on one plot. As noted in Section 19's top priority item, this is one of the two changes from recent sessions that has not yet been confirmed on the user's real machine — that confirmation should happen early in any resumed session, before building further on top of Match Lookup or Team Lookup.
- Confirm the Stage 13 changes are present: `requirements.txt` should list only `matplotlib` (no `numpy`, `scipy`, or `pandas`); `calculations/distributions.py` should have no `scipy` import and should have a private `_normal_pdf()` function instead; `ui/match_lookup_tab.py` and `ui/team_lookup_tab.py` should have their `matplotlib` imports inside `_draw_alliance_chart()` / `_draw_score_chart()` respectively, rather than at module level. As noted in Section 19's top priority item, this is the other change (alongside Stage 12's chart) that has not yet been confirmed on the user's real machine — confirm the app opens noticeably faster and that both charts still render exactly as before.
- If asked to re-validate Adjusted Score Variance specifically (e.g. as part of deciding whether to revert Stage 6 per Section 10.4): call `calculations.score_variance.build_adjusted_score_variance()` directly (it still exists and works, it's

just not wired into pipeline.py anymore) and compare against FORMULA_EXPORT's "Adjusted Score Variance" sheet exactly as Section 9.5 describes.

- Cross-reference this PDF's Section 6 (calculation logic), Section 9 (Stage 5 bug fixes), Section 10 (Stage 6 experiment), Section 11 (Stage 7 expected-score bug fixes), Section 12 (Stage 8 UI changes), Section 13 (the bookmarked Google Sheet validation status), Section 14 (the Stage 9 import-mixing bug fix), Section 15 (the Stage 10 Team Lookup chart), Section 16 (the Stage 11 Team Lookup panel cleanup), Section 17 (the Stage 12 Match Lookup chart), Section 18 (the Stage 13 startup-performance optimization), and Section 20 (Stage 4 base import detail) to see exactly which Python function or UI behavior corresponds to which sheet, which judgment calls were already made, and why — without needing to re-derive any of the analysis from scratch.

The single most valuable next action for whoever picks this up is the one flagged at the top of Section 19: confirm, together, Stage 12's Match Lookup "Score Distribution" chart and Stage 13's startup-performance changes on a real display. Team Lookup's Stage 10 chart and Stage 11 cleanup have already been confirmed working by the user on their own machine (Sections 15.3 and 16.4) — Match Lookup's Stage 12 chart and Stage 13's startup-performance changes are the two remaining pieces of recent work that have only been validated headlessly and still need that same real-machine confirmation.

End of summary. Generated to allow this project to be resumed with no access to the original chat history. This is the v7 revision, updated at the end of the Stage 13 (startup performance optimization) session to remain accurate and complete through Stage 13.